

Diver Project

Utilizing Machine Learning Classification for Enhancing
Audio Clarity for Divers
Group 727

7th semester project



Page left intentionally blank



Electronic Systems
Aalborg University
<http://www.aau.dk>

AALBORG UNIVERSITY

STUDENT REPORT

Title:

Utilizing Machine Learning Classification for Enhancing Audio Clarity for Divers

Theme:

Machine Intelligence

Project Period:

Fall Semester 2024

Project Group:

727

Participant(s):

Emil Kirk Andersen
Jónas Á Hálmörk Joensen
Jonas Pedesk
Kasper Mølgård Jørgensen
Kazi Towhidur Rahman
Kristoffer Elmann Gustavussen

Supervisor(s):

Dorte Hammershøi

Copies: 1

Page Numbers: 61

Date of Completion:

December 13, 2024

Abstract:

This project sets up a machine learning model for classifying different parts of a diver's audio signal at Nordsøen Oceanarium. The purpose of the model is to see if the audio levels of the characteristics of the voice of the diver can be controlled separately from the undesired sounds, as this may improve the experience of the audience. Audio recordings of the diver were explored. After having the audio labeled, high classification accuracies could be achieved across different machine learning models. It was possible to use the predictions to segment audio files based on four categories, but final tests showed that varying levels of unintended information loss occurred when attenuating the audio.

Contents

Preface	v
List of Acronyms	vii
1 Introduction	1
2 Problem analysis	2
2.1 Nordsøen Technical Setup	2
2.2 Audio and Sound Processing Considerations	4
3 Machine learning theory	7
3.1 Feature Extraction Algorithms	7
3.2 Machine Learning Models	10
3.3 Feature Selection	12
3.4 Segmentation	15
3.5 Classification Performance Measurements	18
3.6 Dimensionality Reduction	22
4 Requirements	24
5 System Development	25
5.1 System Overview	25
5.2 Development Practices	29
5.3 Preprocessing	30
5.4 Model Training	34
5.5 Model Application	39
5.6 Model Optimization	41
6 Test	44
6.1 Model Performance Test	44
6.2 Audio Prediction and Attenuation Test	47
7 Discussion	50
7.1 What was Achieved	50
7.2 System Limitations	52
7.3 Further Work	53
7.4 Worth Considering	55
8 Conclusion	57
Bibliography	58

Preface

This project was written during the 7th semester of the program Electronic Systems, at the Department of Electronic Systems at Aalborg University. The report assumes a rudimentary knowledge level of concepts within signal processing and machine learning.

Proper citation of sources is maintained throughout the report using the Modern Language Association's (MLA) reference system. Each reference is assigned a numerical identifier corresponding to its position in the list of sources. Figures, tables, and attachments are similarly referenced in the text using corresponding numerical identifiers.

The group would like to thank the project supervisor Dorte Hammershøi for her guidance throughout the writing of the project. Furthermore, the group would like to thank Nordsøen Oceanarium for their collaboration on the project. Lastly, the group extends their gratitude to Nicoleta Diana Radu, who previously tackled a related project and made her research data, recordings, and progress available to the group. Nicoleta's prior work was used as a basis of data for creating a Machine Learning model. The group used this to make new sets of annotated data, which was used for supervised machine learning. Furthermore, it was used to get an understanding of what might be achieved in the project, albeit this project uses a different approach, such as differing in the choices of the classes of audio segments and with the intention of understanding the project from a technical perspective, rather than from a perspective of visitor engagement and perception [1].

Aalborg University, December 13, 2024



Emil Kirk Andersen
<emiand21@student.aau.dk>



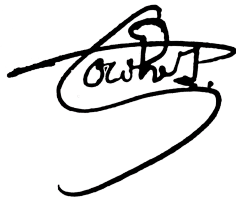
Jónas Á Hálmørk Joensen
<jjoe21@student.aau.dk>



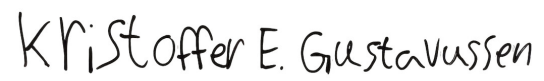
Jonas Pedesk
<jpedes20@student.aau.dk>



Kasper Mølgård Jørgensen
<kasjor21@student.aau.dk>



Kazi Towhidur Rahman
<Krahma24@student.aau.dk>



Kristoffer Elmmann Gustavussen
<kgusta21@student.aau.dk>

List of Acronyms

"MBS"	"Mix-Bubbles-Speech". 28, 29, 34, 36–38
BFE	Backward Feature Elimination. 14, 35
DFT	Discrete Fourier Transform. 7
FFS	Forward Feature Selection. 14
FN	False Negative. 37, 51
FP	False Positive. 37, 51
GBDT	Gradient Boosted Decision Trees. 12
HMM	Hidden Markov Model. 11
KNN	k-nearest neighbor. 36
LDA	Linear Discriminant Analysis. 22, 23, 34–38, 41
LOOCV	Leave-One-Out Cross-Validation. 19, 20
LPC	Linear Prediction Coefficient. 31
LPCC	Linear Prediction Cepstral Coefficients. 9, 31
LSTM	Long Short-Term Memory. 10
LSVC	Linear Support Vector Classification. 34–38, 40, 41, 50, 51
MFCC	Mel-Frequency Cepstrum Coefficients. 7–9
ML	machine learning. 7, 12–15, 25, 42, 51, 52, 54, 56
PCA	Principal Component Analysis. 22
RF	Random Forest model. 12, 15, 34–38, 41–43, 46, 50, 51
RFE	Recursive Feature Elimination. 14, 35, 41, 42, 44, 51, 55
RFECV	Recursive feature elimination with cross-validation. 46, 51

RNN	Recurrent Neural Network. 10
STE	Short Term Energy. 8, 9
SVC	Support Vector Classification. 34–38, 41, 43, 45, 46
SVM	Support Vector Machine. 10, 12, 42
TP	True Positive. 37
VAD	Voice Activity Detection. 1, 2, 6
ZCR	Zero-Crossing Rate. 8, 9, 17

1: Introduction

Finding out how to separate a desired sound signal from background noise is a long-standing research area. While noise-reduction and noise-canceling solutions already exist, there are practical cases in which it is difficult to distinguish noise from the wanted part of the audio. While useful datasets can be found for some sound-and-noise situations, there are also situations in which accurate, existing data cannot be found, meaning that it is not possible to train a model for blocking noise in the given situation. This necessitates the process of manually obtaining data and labeling it to do analysis of the characteristics of the desired parts of the signal and background noise to create a system that blocks out the undesired noise.

This project focuses on a practical problem at Nordsøen Oceanarium in Hirtshals. Here, a daily live show takes place in which a diver feeds fish while also communicating with a presenter outside who guides the show. The spectators hear the diver, whose speech is occasionally interrupted by the sound of breathing, since the diver's microphone is placed inside their diving mask. Voice Activity Detection (VAD) may be useful to distinguish pure talking from the sound of breathing.

To work on diminishing the noise level of the breathing of the diver, a way should be found to control the level and characteristics of the breathing sounds separately from the voice level, requiring a new VAD to be designed for this special case.

How can a machine learning system be constructed to segment and classify the diver's audio signal?

2: Problem analysis

In traditional VAD, the goal is to filter out some background noise. In a common use case, the noise is purely produced by something other than the source of the signal of interest. For the microphone that captures the diver's voice, this is not exactly the case, as the primary noise element, the breathing of the diver, is also produced by the diver.

This chapter explores these challenges with the aim of finding the approach that best mitigates the problem that Nordsøen Oceanarium faces.

2.1 Nordsøen Technical Setup

This section briefly explores the technical setup for audio processing at Nordsøen Oceanarium, aiming to get a better understanding of the equipment and how it potentially affects the diver's audio.

In the audio mixing room, it is possible to change the audio settings, and there is a list of presets available designated to each diver.

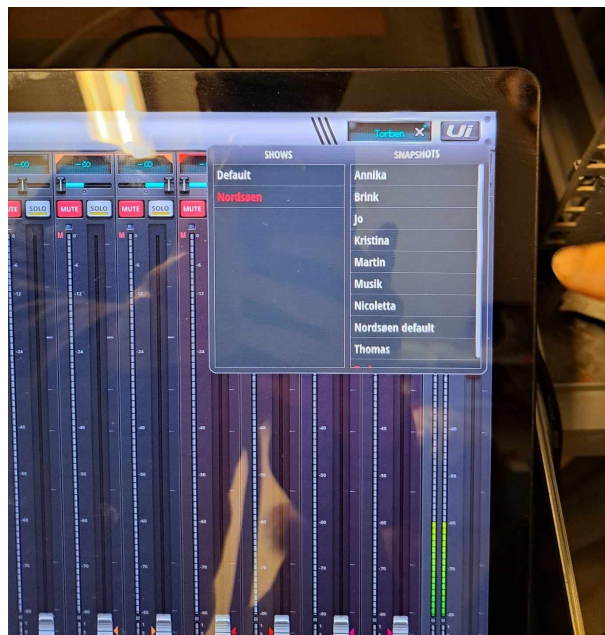


Figure 2.1: Image of audio mixing interface with different settings for different divers

Figure 2.1 shows the interface of the mixing console, which is a Soundcraft Ui16 Digital Mixer. Figure 2.1 a shows the interface of the mixing console through a web interface, which can be accessed through an IP-address.

It is also worth considering the other technical equipment, and the physical environment in which it is used. Firstly, the diver's mask is considered.



Figure 2.2: Image of diving mask used at Nordsøen Oceanarium



Figure 2.3: Image showing connection cable between diving mask and other equipment

Figure 2.2 shows a diving mask used at Nordsøen Oceanarium. In Figure 2.2, it can be seen that salt formations have started to form on the inside of the mask. This makes it evident that some amount of water gets into the mask. Even though the mask is sealed, water may spill into it when it is taken off outside the aquarium. The air above the aquarium has a high salt and water content, meaning that salt deposits can form on things in the immediate area. Figure 2.3 shows a cable that connects the microphone in the diving mask with the audio processing system. In Figure 2.3, it can be seen that salt formations have formed on both cables. When visiting the Oceanarium, the group was informed that the salt and moisture could be an issue, as it can enter the electronics and affect them. It was also told that the divers can experience an issue with cable connection, where the cable, shown in Figure 2.3, cannot make a close seal, because something is stuck in the connector. In part, this can cause an issue where the condition of the cable will continuously get worse every time that it is used. It is also an issue, as impurities on the metal may slowly compromise the connection. On the 30th of October, at 11:44 am before the show, it occurred that something was stuck in the cables' connector before the show. It could not be fixed in time so the diver could not talk during the show. The cable could not be replaced, because their other cable was sent for repair.

It is therefore worth noting that this system will not be able to solve all issues that may be experienced when using the system. The focus has been on when the system is fully functional. Some of the previously mentioned concerns may also affect the audio quality of the signal, which, depending on how this problem is solved, will

need to be taken into account.

2.2 Audio and Sound Processing Considerations

Some considerations can be made about audio and sound processing to understand what may be relevant for this project.

Physically, sound can be considered as waves that produce a change in the pressure of the medium it travels through. Traveling sound waves will produce acoustic energy in the directions that they propagate, which may be used for analysis.

In this project, sound is considered from the physical perspective. The variations of sound in the physical medium is recorded through electronic hardware, after which it can be analyzed and described.

Considerations may also be made about the sources of audio in the project. In speech recordings, both the production of vocal signals and the breathing patterns play a role in the created audio characteristics. During non-speech, a typical breathing cycle consists of 40% inhalation and 60% exhalation in regular breathing [2]. However, during speech, this balance shifts, where inhalation may occupy as little as 10% of the cycle, and exhalation takes up 90%. Here, the extended exhalation phase is required for speech, as air is used to produce sound. Depending on a speaker's need for air, the breathing will have variations in inhalation and exhalation patterns. It may be the case that a professional diver at work has dynamic breathing patterns, since the job requires physical activity, which can cause irregular breathing patterns. E.g., the diver may take quick breaths or speak in shorter bursts. It should be noted that speech can also be produced during inhalation rather than exhalation, leading to differences in the measured signal. This may also cause a situation where there are mixed audio samples of the diver both talking and breathing.

This is to exemplify that in practice, the audio classification process of this project may not be straight forward. Audio created by humans is composed of both vocal signals and breathing. As the content of breathing cycles varies, and the nature of breathing cycles during speech production can vary, it is likely that the characteristics of audio recordings of "breathing" and "talking" may vary.

Visit to Nordsøen Oceanarium

These questions were explored further on the 30th of October when the group members visited Nordsøen Oceanarium and talked with Diver and Chief Biologist Kristina Ydesen, who exemplified the issues that can be experienced during a diving session, and the audio variations that may occur during a dive. Kristina noted that during a show, they may switch between Danish and German since the Oceanarium receives many guests from Germany. When speaking German, the divers can experience that their tone of voice may be lower than when speaking Danish. Also, the lengths of

their breaths may vary naturally. Some sentences are long, and some sentences are short. Also, when interacting with the audience or the presenter, it may also be experienced that their breaths have different lengths depending on how long it takes to wait for a response. Furthermore, Kristina noted that the divers do experience physical exertion. When diving, the divers swim with a combined cable with multiple purposes. The cord feeds air into the diving mask, but also includes the cables for the diver's microphone, headset, and camera. The issue of the cable bundle is that it creates a lot of buoyancy, meaning that the divers must use a lot of force to stay in position.

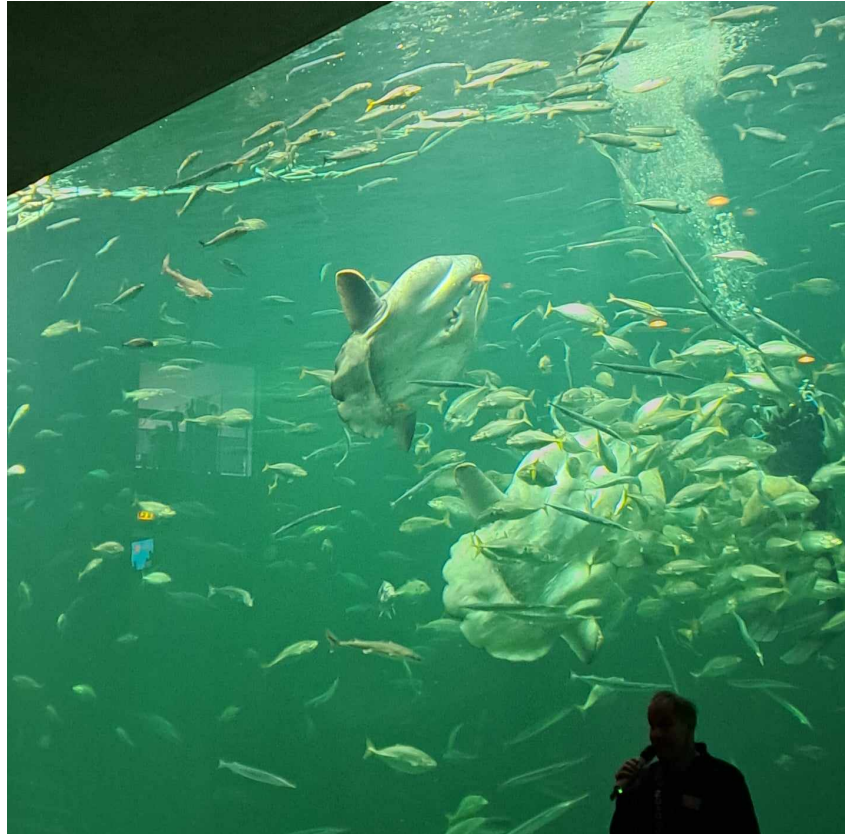


Figure 2.4: Image showing a diver, Kristina, diving in the large aquarium at Nordsøen Oceanarium. The cable bundle can be seen floating on top of the water surface, creating buoyancy.

Figure 2.4 was taken on the 30th of October 2024. It shows Kristina diving for the daily feeding session of the aquarium. Figure 2.4 shows that while the diver is in the aquarium, the cable experiences lift, with much of the cable lying on the surface of the water. This shows a situation in which the diver uses physical force against the cable, which can lead to differences in breathing patterns.

Data Capture Properties

This project is based on audio recordings that were made for a previous project tackling a similar task. Here, a short description will follow to describe the data capture and its properties [1].

The diver's audio was collected using a digital audio recorder connected to the output of the diver's microphone system. It should be noted that some of the audio recordings were recorded at 96 kHz, while later recordings were made at an adjusted sampling rate of 44.1 kHz. The author made the following explanation of her choice [1]. The human auditory system spans from 20 Hz to 20 kHz. This means that a sampling rate of 96 kHz greatly exceeds the frequency needed to capture audio within the relevant spectrum. The decision was made with regard to the Nyquist-Shannon sampling theorem, explaining that the signal can be recorded without aliasing if the sampling rate is at least twice as large as the largest frequency component of the signal. This means that the later recordings had a Nyquist frequency of 22.05 kHz, which is just slightly above the human auditory range. It was argued that this represented the relevant perceivable audio frequencies while minimizing redundancy and optimizing the file size. The files recorded at 96 kHz were later downsampled to 44.1 kHz [1].

Some things should be considered when processing the data. Namely, the downsampling removed samples from the recordings but left the length unchanged. This causes an inherent limitation to the project, as it will not be possible to see which performance differences can be obtained by working with 96 kHz data. That is, the downsampling might have removed noisy data, or it might have removed data that could help to classify the audio - it cannot be known. The frequency response of the microphones is not known, but it would be safe to assume that it does not cover much more than the auditory human hearing range. As such, this might not be an issue.

Voice Activity Detection (VAD)

VAD is a technique used to identify the presence or absence of speech in a given signal. It is used to classify and segment audio signals into regions with and without speech. VAD systems distinguish speech from non-speech through different feature extraction methods and through different decision-making methods that can be either based on statistics, thresholds, or machine learning [3, pp. 185–201].

3: Machine learning theory

This chapter covers the basic theory of machine learning that is used in the project. This includes ML models, feature selection and segmentation.

3.1 Feature Extraction Algorithms

Feature extraction algorithms are applied to the data before it is put into the machine learning algorithm. These are used to ensure that the machine learning algorithm focuses on specific characteristics of the input signal. It is worth considering which characteristics are the most relevant for distinguishing different classes. Some relevant algorithms for this are briefly presented in the following subsections.

Mel-Frequency Cepstral Coefficients (MFCCs):

Mel-Frequency Cepstrum Coefficients (MFCC) are widely used in speech and audio processing to represent the short-term power spectrum of a sound. It utilizes the theory of the Fourier transform, the mel spectrogram (mel scale), and the cepstrum, the cepstrum being the inverse of a log-spectrum. MFCC has been widely used in applications regarding the detection of speech, speakers, and music. [4, p. 13][5]. The whole process for performing MFCC can be seen on Figure 3.1:

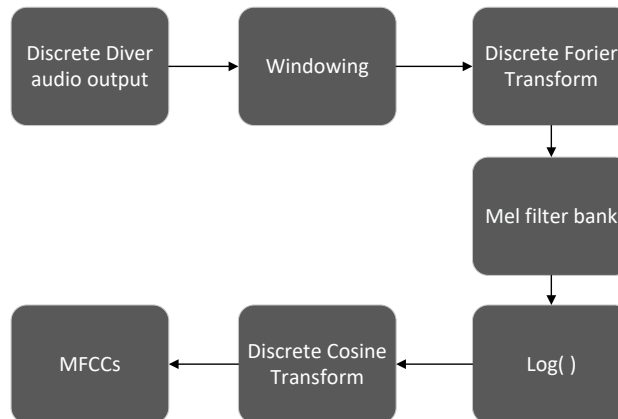


Figure 3.1: Diagram of how the MFCCs can be found.

It creates coefficients based on the power spectrum of the input signal. These coefficients can be used to make judgments on the signal. Assuming windowing has already been done, the DFT is then performed. The mel filter bank is applied to the

spectrum of the system. The logarithm of this is then calculated, and lastly the Discrete Cosine Transform is done. This gives the MFCCs [4, p. 13] [5].

Zero Crossing Rate

The Zero-Crossing Rate (ZCR) is defined as the rate of change of the sign of the signal. Calculating the ZCR for an audio frame will give different results from other audio frames, depending on the change in the frequency content. One use case for ZCR is to determine if a speech audio frame is voiced (when the vocal cords are used), unvoiced (consonant sounds, for example), silent, or contains noise. The ZCR is higher for unvoiced portions of speech than the voiced portion [4, pp. 4–5]. Ideally, the silence portion should have a ZCR of zero in the absence of noise [4, pp. 4–5]. Furthermore, as breathing is an unvoiced sound and speech contains voice parts, it should have a higher overall ZCR than speech segments.

Short Time Energy

Another way to evaluate a signal is to look at the energy of the signal in short bursts. When talking about the energy of the signal, what is really calculated is the mean of a short frame of the signal. This energy can be calculated using the following expression:

$$E_n = \sum_{m=n-N+1}^n [x(m) \cdot w(n-m)] \quad (3.1)$$

where: $w(n-m)$ = The chosen window

n = Sample that is at the center of the analysis window

N = Window length

When considering speech, the Short Term Energy (STE) is generally higher for voiced parts of the signal than the unvoiced parts [6, p. 1]. This could be useful for differentiating between speech parts and breathing parts of an audio signal.

Spectral Centroid

The spectral centroid is a measure used to describe the center of mass in a spectrum. It indicates where the center of the spectral energy is placed on the frequency axis[4].

Spectral Rolloff

The spectral rolloff point is defined as the point where 85 % of the signal energy is contained below the given frequency. It is used for separating low frequency components from higher frequency components in the given signal[4, 7].

Spectral Contrast

Spectral contrast is defined as the decibel difference between peaks and valleys in a spectrum. This contrast will be more noticeable for human speech due to its tonal nature.

[8].

Spectral Spread

Spectral spread, also called spectral dispersion, is a feature related to the bandwidth of the signal. The spectral spread is the average spread from the center frequency in the spectrum of the given signal [9, p. 277]. Noise signals and environmental sounds generally have a wide spectral spread compared to speech signals, which are generally more narrow [4, p. 11].

Spectral Skewness

Spectral skewness is a measure of how much frequency content there is above and below a given center frequency. A skewness equal to zero describes a symmetric distribution. A negative skewness indicates more energy to the right side of the spectral distribution, while a positive skewness describes more energy components to the left [4, p. 11].

Linear Prediction Coefficients (LPC) and Linear Prediction Cepstral Coefficients (LPCC)

These are two related feature extraction algorithms that have seen extensive use in the field of Automatic Speech Recognition. They both build on the concept of linear predictive coding and making assumptions of how voiced sounds are produced. LPCCs take it a step further and, similarly to MFCCs, find the cepstral coefficients for the LPC [10].

Based on this, the following algorithms were chosen to be part of the implementation: MFCCs, ZCR, STE, spectral centroid, spectral rolloff, spectral contrast, and LPCCs. Of these, MFCCs and LPCCs are of special interest, given they have been used extensively in speech recognition [4, 11].

3.2 Machine Learning Models

Machine learning models are the specific algorithms used to look for patterns in the input, and can be used to build a classifier for predicting the output. There are many different models available, all with different advantages and disadvantages. Some of the algorithms can be found below.

Long Short-Term Memory (LSTM) Networks:

Long Short-Term Memory (LSTM) is a type of Recurrent Neural Network (RNN), which has proven useful for sequential data like audio signals, often applied in speech recognition and music generation. One way it could be used is to detect the endings of sentences, which can serve as an indication of when a person may start breathing [12, 13].

Support Vector Machines (SVMs):

A Support Vector Machine (SVM) is a supervised ML-model that can be used for classification tasks in audio analysis, such as distinguishing between different types of sounds. It also has the added advantage that it handles being trained on small datasets better than most other algorithms. SVM can be used as a linear classifier, where the premise is to find an optimal line or hyperplane that maximizes the distance between the classes in a space. This is done by maximizing the margin size using support vectors [14]. This can be seen illustrated on figure 3.2.

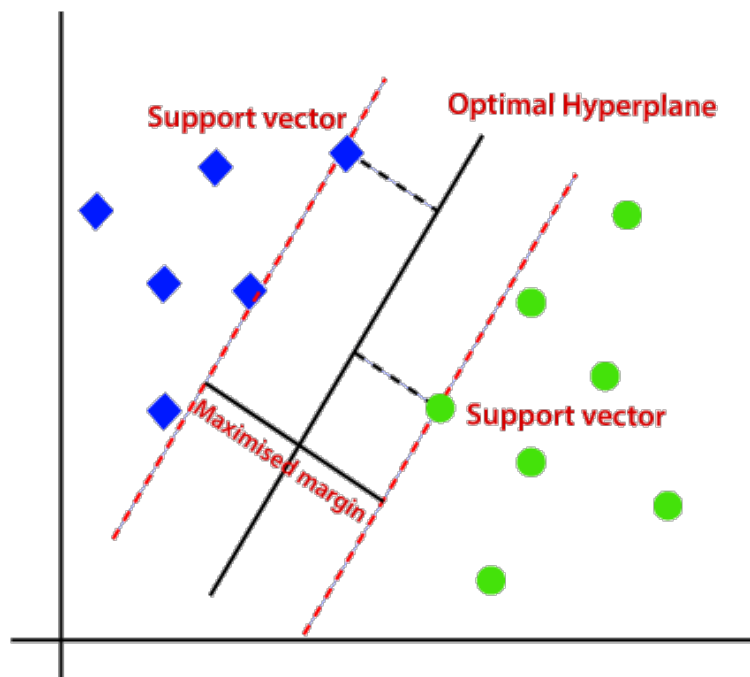


Figure 3.2: A graphical illustration of SVM [15]

Hidden Markov Model (HMM)

Hidden Markov Models (HMM) are commonly used in speech recognition to model the sequence of speech sounds. It utilizes the theory of Markov chains, where it uses the previous "state" to find out what the next state is. The "hidden" part is the Markov chains that the system tries to estimate [16].

Decision Tree

A decision tree is a non-parametric supervised learning algorithm, and it is used for both classification and regression tasks. It has a hierarchical tree structure that consists of a root node, branches, internal nodes, and leaf nodes[17].

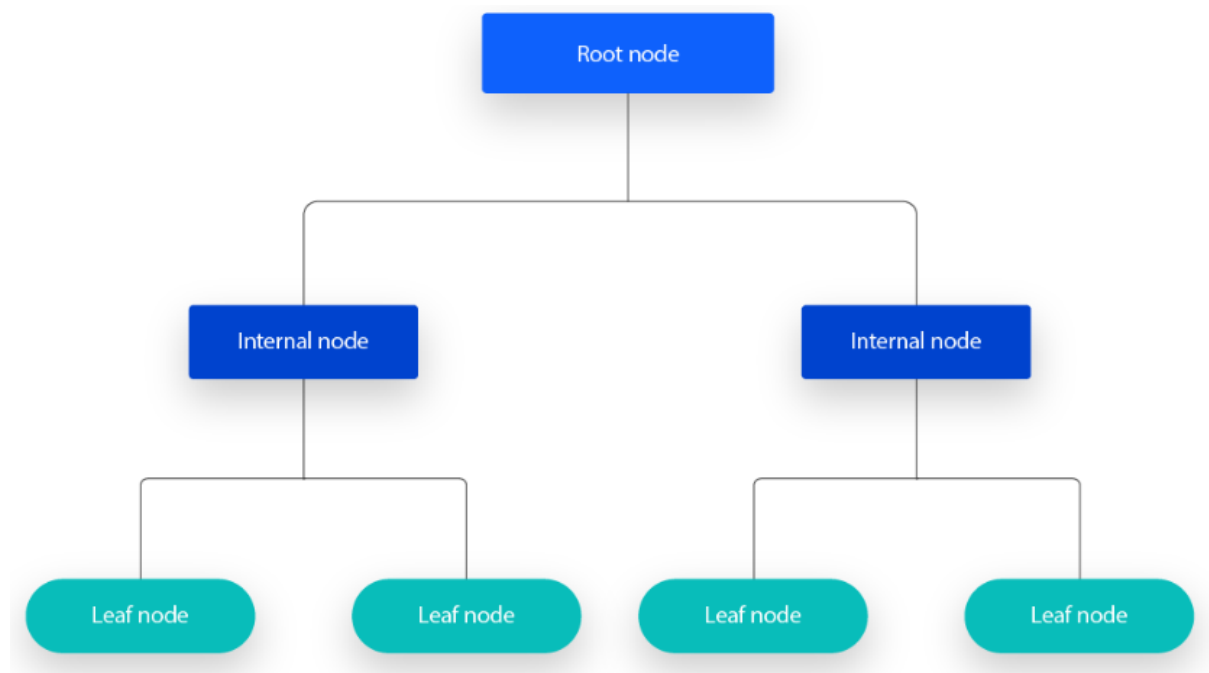


Figure 3.3: General form of a decision tree [17]

One method of choosing the best attribute at each node is to maximize the information gain. Information gain is calculated as a difference in entropy. Entropy refers to the measure of randomness of data. Entropy values can fall between 0 and 1. If all samples in the data set belong to one class, the entropy will equal 0, and if the samples are split equally between the classes, the entropy will be 1 [17].

Splitting the sample set based on a feature creates two subsets for which the entropy can be calculated. If a lower entropy is obtained, the information gain is higher, and the decision tree will proceed with that option.

An advantage of decision trees is that they are easy for people to interpret due to their use of boolean logic and hierarchical structure. This hierarchical nature also makes it easier to see which attributes are most important. Decision trees can be

prone to over-fitting when increasing their complexity. Also, due to the "greedy search" approach during construction, they can be more computationally expensive to train compared to other algorithms. [17]

Random Forest Classifier

The Random Forest model (RF) is based on decision trees. It creates what is called bootstrapped datasets. These are randomized subsets of the original data. Each dataset is then used to create a decision tree, and these trees are then run in parallel, thus creating a forest. The result of each is then aggregated to give the final result [18].

Gradient Boosted Decision Trees (GBDT)

Gradient Boosted Decision Trees (GBDT) is also based on decision trees. It improves its performance by optimizing the gradient of the loss function. It is based on the concept of boosting. By placing decision trees sequentially, the next tree in line can be used to help correct the mistakes of the previous tree. GBDT is prone to overfitting and does not handle outliers well [19].

While all these algorithms can be used for the purpose of classification, not all of them would be ideal. The specific use case requires the classification and filtering of the signal, ideally in real-time. It is also worth keeping in mind that the audio source is a mix of speech and "diving/breathing sounds" as it can also have an impact on which algorithm is chosen. Given these machine learning models, it was decided to test and compare the performance of SVM and RF.

3.3 Feature Selection

It may not be clear which grouping of feature extraction algorithms will give the most accurate result. Features help to point out what in the given signal the machine learning model should be focused on. This does not mean that the patterns that the model finds will be relevant for the classification goal. In the worst case, a pattern recognized by the machine learning algorithm could make the model less accurate while adding to the number of computations needed. This is the motivation for performing feature selection [20]. Feature selection techniques take a given list of features and test to find out which subset of the features gives the highest accuracy for the model. To do this, a list of features must be chosen. The individual algorithms are chosen based on whether it is believed they could be relevant for the given use case. It is important to ensure that all feature algorithms add to the accuracy of the model. If not, then removing them will improve efficiency and can improve accuracy [21]. An overview of these methods can be found in Figure 3.4.

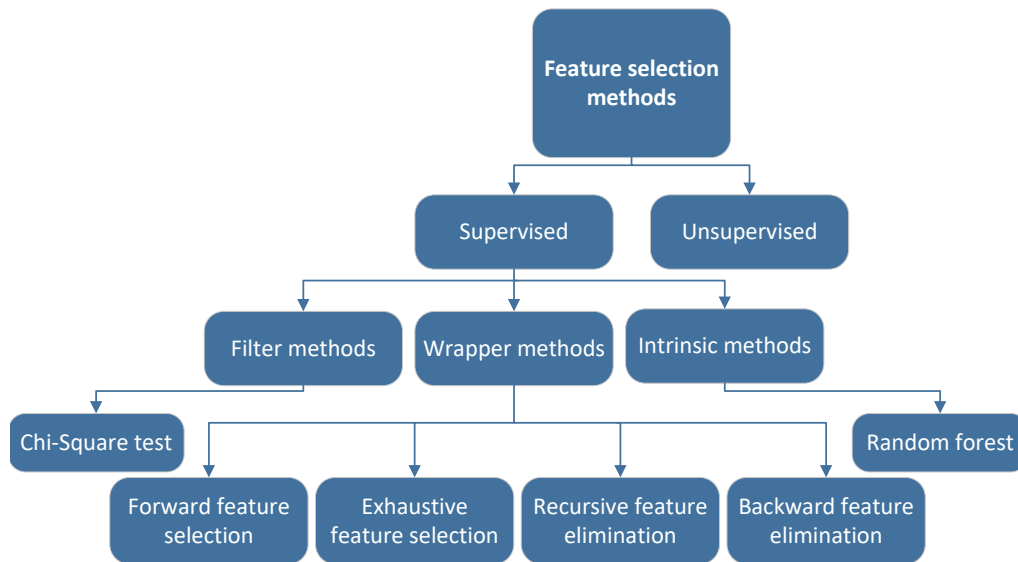


Figure 3.4: An overview of the discussed feature selection methods

First, these methods are split into two categories: Supervised and unsupervised feature selection [21].

Unsupervised Feature Selection

Unsupervised feature selection, similarly to unsupervised machine learning, refers to methods that can be used without labeled training data [20]. In this project, labeled data will be used. However, the clustering method has been used to visualize the audio data and figure out a possible number of classes. This topic is further discussed in subsection 5.3. Besides this, the unsupervised method hasn't been used in this project, and thus, the topic will not be explored further.

Supervised Feature Selection

Supervised feature selection, similarly to supervised machine learning, relies on labeled training data. It utilizes the labeled data to figure out which features are most relevant. These can be split into a couple of different categories [20, 21] :

Filter Methods

These methods are often chosen due to their efficiency and speed. They are performed without keeping the ML model in mind. They rely on statistical analysis for evaluating the relevance of each feature extraction algorithm. The Chi-square Test is one method used for evaluating this. It is used to evaluate the correlation of features to the desired output [22]. Each algorithm is evaluated individually, and ranked based on their performance. The most relevant one is then chosen. The disadvantage with this method is that it is not as good as some other methods for finding the

most accurate feature subset. It also does not take the machine learning model used into account [20, 22, 23].

Wrapper Methods

Rather than evaluating each feature extraction algorithm individually, wrapper methods take the list of algorithms and iteratively test subsets to find the best combination. This tends to provide a higher accuracy for the system, at the cost of being more intensive than the filter methods. Below are a couple of these wrapper methods [20].

Forward Feature Selection: Forward Feature Selection (FFS) iteratively finds the features that contribute the most to an improvement in the desired output and adds that feature to the model. This is repeated until the addition of another feature no longer improves the model [23].

Backward Feature Elimination: Backward Feature Elimination (BFE) starts by using the full list of features, and then it iteratively removes the features that contribute the least to the accuracy of the system. This is done for one feature at a time until the number of features has been reduced such that a desired number of features is reached, or it no longer improves the model [23]. To do so, BFE ranks the importance of features by considering how important they are to achieve a desired result. By eliminating the variables that have the least predictive power, BFE may help to avoid data over-fitting. That is, if too many features are kept, a model may incorporate patterns that are either weak or misleading [20].

Exhaustive Feature Selection: Also known as exhaustive search. This method creates all the possible combinations of features and compares them to find the optimal solution. This is not ideal, as it has to go through every possible iteration, making it computationally expensive [24].

Recursive Feature Elimination: Recursive Feature Elimination (RFE), as the name suggests, recursively creates subsets of features and compares their performance, starting by using the full set of features. Depending on the performance of the model, features are removed until the subset is reduced to a given size [20, 21, 23]. RFE starts off by considering all the features of a training dataset. Afterward, it removes features until a desired amount of features remains. In doing so, RFE recursively fits a model for every step when it removes features. In re-fitting the model, it determines which features are the most important in the model. In doing so, RFE reduces the dimensionality and complexity of a dataset while trying to keep its most important features. Where RFE differs from BFE is that RFE re-fits its model for each iteration, whereas BFE does not. This may mean that RFE can find a better feature subset than BFE, as the model is iteratively fitted to find the best feature subset.

Intrinsic Feature Selection

Some machine learning models can intrinsically apply feature selection into their model. This means that they perform feature selection automatically as part of the machine learning model. Examples of this are models based on decision trees, such as the Random Forest model [20, 21].

While there are many advantages with utilizing these methods, there are also some issues worth addressing. These methods are prone to over-fitting when using small datasets. It also becomes increasingly computationally heavy as the dimension of the data increases [20]. While these issues are worth noting, the improvements in accuracy can make these methods worth using.

3.4 Segmentation

In the context of this project, segmentation refers to the method used to determine when different parts of the signal should be split into different segments. In this report, two approaches are discussed: adaptive and non-adaptive segmentation. They differ in how they split the data and have their own pros and cons.

Non-Adaptive Segmentation Algorithm

For the purposes of this report, non-adaptive segmentation refers to using segments of a fixed size. Features are extracted for each of these segments, creating feature vectors that correspond to each segment. After the feature vector is calculated for a segment, it is used to classify the segment using the trained machine learning classifier. The segments of these classes would be post-processed differently.

The benefit of this approach is that it should be relatively simple to implement in a real-time situation. Fixed segments have their features extracted, and the audio classes are then predicted, attenuated, and output.

An example of non-adaptive segmentation can be seen below in Figure 3.5. The rapid oscillation is considered noise, and the less rapid oscillations are considered speech, and are therefore of interest.

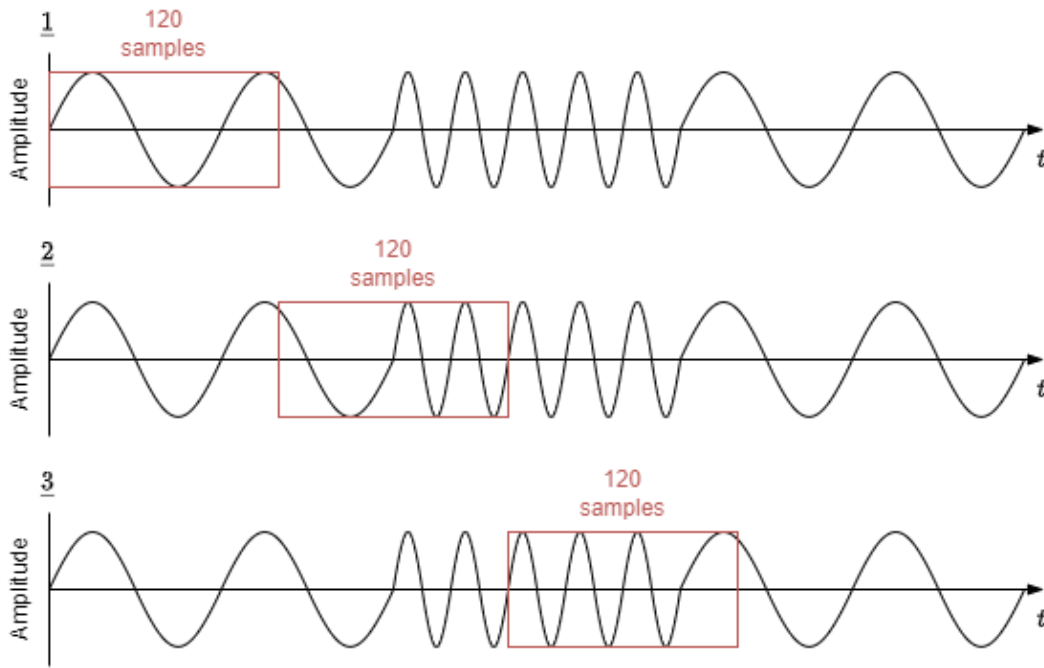


Figure 3.5: An example of non-adaptive segmentation

The specific example in Figure 3.5 calculates the features for 120 samples at the time. When the first window of 120 samples has been processed, the window will be moved onto the next part of the signal. This can be seen in step 2, and here one of the downsides of non-adaptive segmentation becomes apparent. The window lands in the transition state between two signal characteristics, and depending on which features are being calculated, this can lead to misclassification. Either the segment will be seen as noise, and some part of the speech signal is attenuated away. Conversely, if the segment is seen as speech, then the noisy part will not be attenuated. This is seen again in step 3, where it would seem that there is more noise in the window than there is speech, so here the segment would most likely get attenuated, thus losing some speech. These types of problems can be partly mitigated by appropriately sizing the evaluation window.

Adaptive Segmentation Algorithm

For the purposes of this report, adaptive segmentation refers to the utilization of changes in the feature vectors of two windows running alongside each other to determine when to segment the signal. Once this difference between the two vectors reaches some defined threshold, a split is made.

If a split is not made, then there are a few different ways to continue. One way would be for window 1 to incrementally increase, taking over part of window 2, and then recalculating the features. Another option would be increasing the size of window 1 and, instead of taking part of window 2, pushing that same amount ahead in the time-series. Then the features are once again calculated. Below, in Figure 3.6, an

example of this second definition can be seen. This is a simplified version, dealing only with ZCR. Here, the rapid oscillation again indicates noise, and the less rapid oscillation indicates speech.

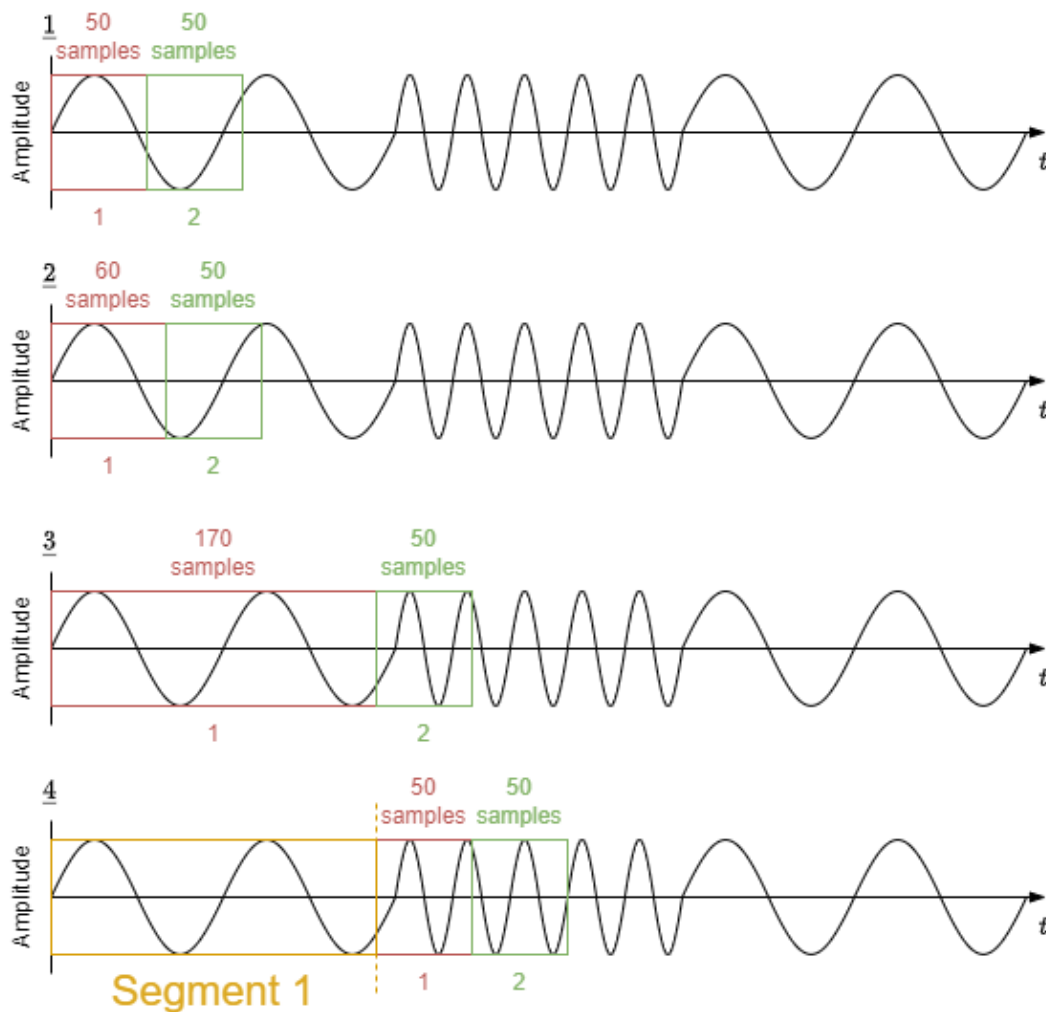


Figure 3.6: An example of adaptive segmentation based on Zero-Crossing Rate (ZCR)

What is shown in Figure 3.6 is an example of adaptive segmentation dealing with the ZCR of the windows. While this is only using a single feature, the same idea can be applied to a feature vector. Here, the algorithm is using two windows, with window 2 being pushed ahead by 10 samples after each iteration. First, the windows are both initialized to 50 samples, from which the ZCR is calculated. Since the initial calculation did not warrant a break, window 1 increases by 10 samples, window 2 is advanced by 10 samples, and the calculation is made once more. This process continues until step 3, where window 2 has a high enough ZCR that it warrants a break. The break is what happens in step 4, where a segment is created. This segment consists of the 170 samples that were in window 1 up until that point. After the break, the windows are reset to their original size, and the start of the next segment is placed at the end of the previous one.

When looking at step 4, it is apparent that when the segmentation happens, there may still be some part of the speech signal present in the segment containing the noise. This would mean that some speech would be lost, reducing the accuracy of the system. To avoid this type of segmentation error, the feature vector could be extended with more features that are good at detecting speech.

This approach has the benefit that, despite the potential for errors like the one shown in Figure 3.6, the method is still segmenting based on the features of the signal. This eliminates the potential for a window to land in the transition state between two classifications.

The downside to this method would be that a real-time implementation could prove difficult due to the two windows that are being compared continuously with each other.

Going forward, the case of non-adaptive segmentation will be focused on.

3.5 Classification Performance Measurements

To measure how well the system works, the F1-score is calculated. The F1-score is a measurement of the accuracy of the system, and it is calculated based on the precision and recall of the system [25]. The F1-score is calculated as shown in (3.2):

$$F1 - Score = \frac{2 \cdot Precision \cdot Recall}{Precision + Recall} \quad (3.2)$$

Precision and recall are both concepts based on checking all the guesses that the system makes. These guesses are then put into the following categories: True positive (TP), false positive (FP), true negative (TN), and false negative (FN). Using the values of each of these categories, the precision, recall, and accuracy can be calculated as shown in the following formulas, starting with precision in Equation 3.3:

$$Precision = \frac{TP}{TP + FP} \quad (3.3)$$

Precision is a measurement of how often a data point labeled in a certain category is actually from that category [26].

$$Recall = \frac{TP}{TP + FN} \quad (3.4)$$

Recall is a measurement of how often a data point in a given category is labeled correctly by the system [26]. Lastly, the accuracy is calculated using the formula in Equation 3.5:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \quad (3.5)$$

The accuracy compares the amount of correctly labeled data points to the overall number of data points [26].

Based on this, a confusion matrix is constructed. A confusion matrix is a way to visualize the system's accuracy using TP, FP, TN, and FN. An illustration of a confusion matrix can be seen on Figure 3.7.

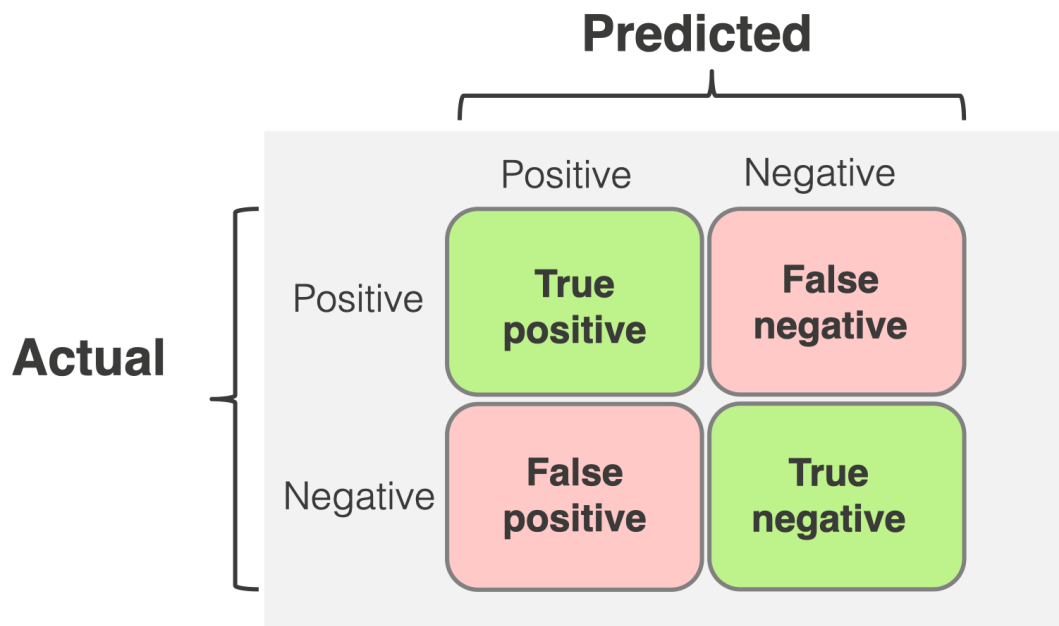


Figure 3.7: Illustration of a Confusion Matrix [26]

Cross Validation

Cross validation is a technique that is used to estimate the predictive performance of a machine learning model [27]. There are several kinds of cross validation. This section will focus on Leave-One-Out Cross-Validation (LOOCV) and k-fold cross-validation, as these can be used as good performance measures for the model that is going to be developed.

The general principle of cross-validation is the same for the two aforementioned cross-validation models. The available data is split up into subsets of data, and some of the data is reserved for validation, while the rest is kept for training. The models then iterate through all the subsets, reserving a new subset for validation each time. These models are usually stratified, meaning that the data in the subsets mimic the

data in the whole dataset. E.g. if there are 100 data points, with two classes, cl_1 with 80 points and cl_2 with 20 points, it is entirely possible, without stratification, to have a subset consisting of all cl_1 or all cl_2 . However, with stratification, each subset will have 8 cl_1 and 2 cl_2 if there were 10 subsets. [27, PP. 3 - 4, line 83-94]

K-fold cross validation is a popular method, as it is simple to understand and yields results that are generally less biased. It works by splitting the data in a K-number of folds, which are subsets of the data, and reserves one for validation. These subsets are of roughly equal size, shown in Equation 3.6 [28].

$$fold\ size = \frac{dataset\ size}{k} \quad (3.6)$$

If it is not possible to create equally sized folds, then one fold will contain the remainder, e.g. if there are 303 examples, and the data is split 10-fold, then one fold will contain 33 examples [28]. 10- and 5-fold cross-validation are among the most common k-fold algorithms. This is because they “have been shown empirically to yield test error rate estimates that suffer neither from excessively high bias nor from very high variance” [29, PP. 209].

The model then iterates through the data k-times, choosing a new fold each time to be the validation subset. The model returns a score for each fold, indicating how good it was at classifying the data in the validation set.

Leave-One-Out Cross-Validation (LOOCV) is a modified version of k-fold cross validation where instead of resampling the data into subsets of multiple examples, the model makes each example its own fold, chooses a fold as validation data, and uses the rest of the data to train the model. This results in a low test error, and a more unbiased estimate of the true prediction error. It results in a higher variance in the data since there is a minimal difference between two different iterations, which differ only by a single example. LOOCV is not recommended for larger (1000+ examples) datasets since it is quite computationally expensive [27, 30].

Below in Figure 3.8 is an example of how Scikit-learn performs K-fold cross validation. Here, the data is initially split up into training and test data, and K-fold cross-validation is used on the training data. After this, the model is subjected to one final evaluation on the test data originally set aside.

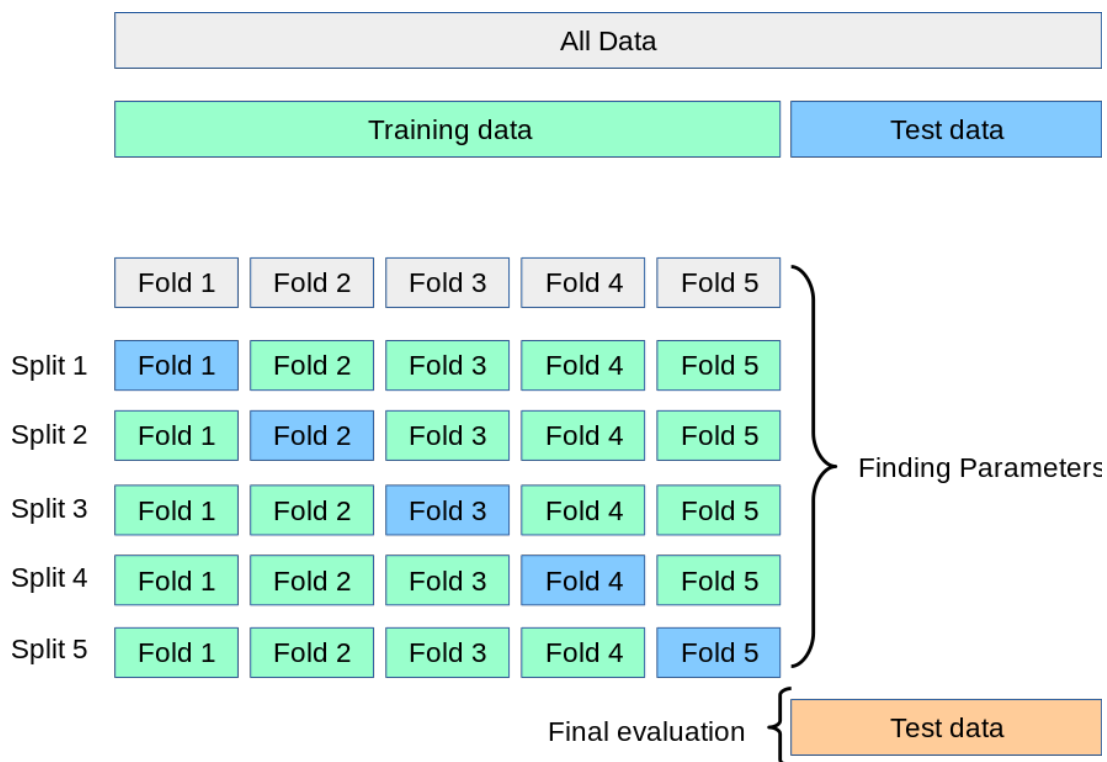


Figure 3.8: An example of k-fold cross validation[31]

When using techniques like cross-validation to estimate the performance of a machine learning model, it is important to have a clear separation between the test and training data. Failing to do this can lead to data leakage.

"Data leakage occurs when information that would not be available at prediction time is used when building the model. This results in overly optimistic performance estimates, for example from cross-validation, and thus poorer performance when the model is used on actually novel data, for example during production." (10.2: Data Leakage, Scikit-Learn, [32])

To avoid data leakage, the split into test and training data should happen first, before any pre-processing. Additionally, it is recommended to separately fit and transform the training and test data to avoid having one dataset influence the other. To do this, a pipeline can be employed. A pipeline is in essence a function of functions, ensuring that the same steps are performed in the same order on separate inputs[32].

3.6 Dimensionality Reduction

Dimensionality reduction refers to techniques used to reduce the number of features. What makes dimensionality reduction different from feature selection is that it does not necessarily preserve the features of the original vector [33]. Instead, it can also transform the original features, possibly combining them, to create new features. An example of this can be seen on Figure 3.9.

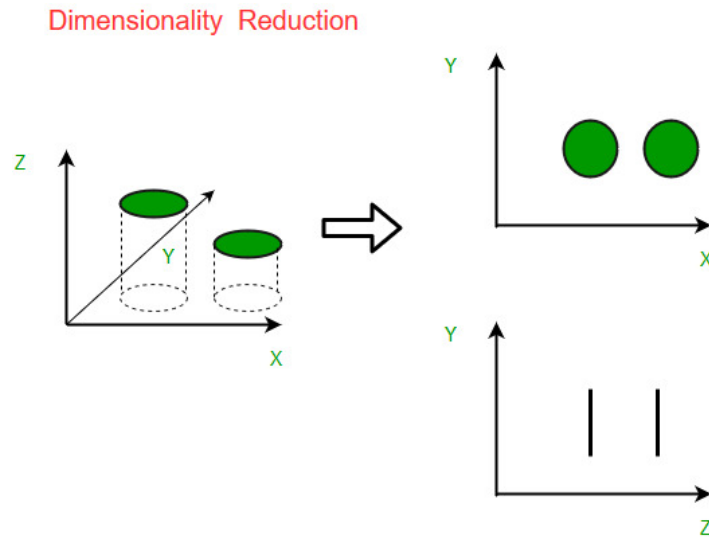


Figure 3.9: This is a graphical representation of three-dimensional data being reduced to two dimensions [34].

Examples of these are Principal Component Analysis (PCA) and Linear Discriminant Analysis (LDA) [33]. While these are considered feature extraction methods, what distinguishes them from the others mentioned previously is that they are often applied to the feature vector, while the ones previously introduced are applied directly to audio signals.

Linear Discriminant Analysis (LDA)

Linear Discriminant Analysis is a technique used to reduce the dimensionality of supervised models. LDA projects the current features into a lower dimension, reducing the dimensionality [35]. An example of this can be seen on Figure 3.10.

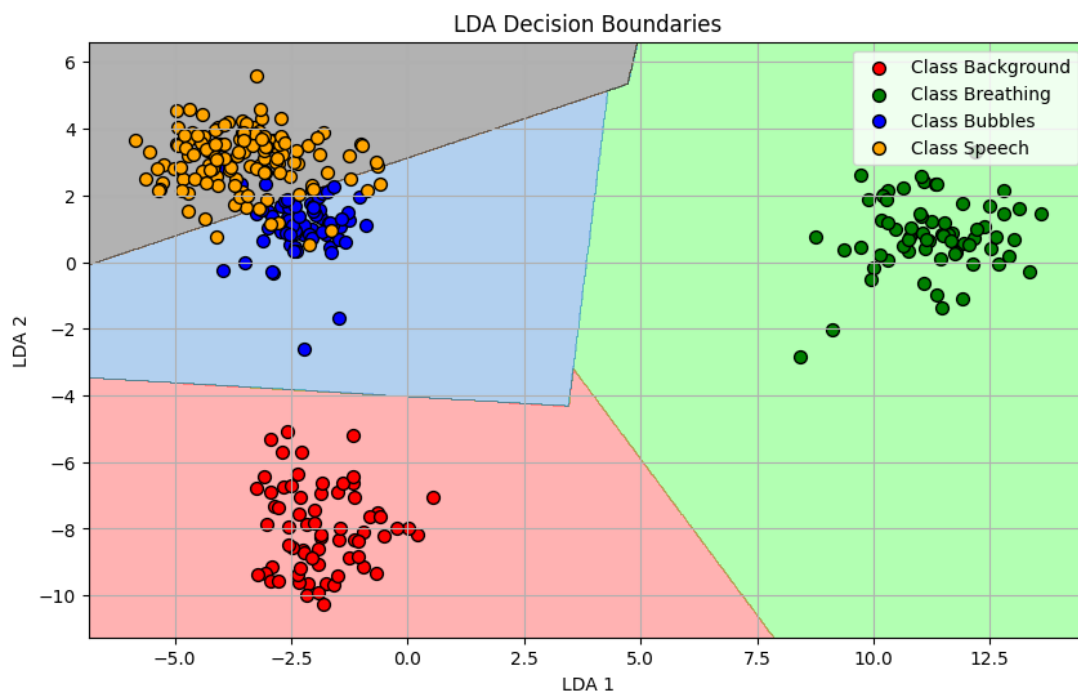


Figure 3.10: This is an example of how LDA can be visualized when applied to a dataset. The lines between the colored sections are called decision boundaries.

The goal of this is to improve class separability while making the model more computationally efficient once trained. This relies on maximizing the separation between the means of the classes and minimizing the variance of each class. The maximum number of dimensions to which it can be reduced is determined by the number of classes. The number of dimensions must be less than the number of classes [35, 36]. Note that LDA can be used as a stand-alone model, but it is often used in combination with another model, where LDA is used to reduce the dimensionality of the data.

4: Requirements

Based on the problem analysis, the following requirements were chosen:

Requirement	Description	Traceability
R1	The classification accuracy for the system should be at or exceed 0.9768	[1]
R2	The precision for the speech detection should be at or exceed 0.97	[1]
R3	The recall for the speech detection should be at or exceed 0.97	[1]
R4	The F1-score for speech detection should be at or exceed 0.97	[1]
R5	The system must not attenuate speech parts of the audio signal.	

Table 4.1: Requirements.

Some of these requirements are based on the work in Nicoleta's report [1]. She achieved these scores for her system, and therefore it has been decided to reach the same results or better.

While the work of Nicoleta is used as a reference, this project and Nicoleta's project are inherently different. Nicoleta's work did preliminary testing, not necessarily aiming to achieve particularly high classification accuracies.

5: System Development

This chapter focuses on the development of the machine learning system. This includes considerations about development practices, pre-processing, model training, model application, and model optimization. The topics of this section will be Feature importance, data labeling, normalization, model comparisons, and data leakage.

The programming language Python is chosen for machine learning and data processing. In part, the reason is that Python is the main language used in the machine learning course of the 7th semester, meaning that the project members have a common ground in understanding how this language can be used for machine learning tasks. Furthermore, Python has an extensive amount of libraries and frameworks for machine learning, scientific data processing, and data visualization.

5.1 System Overview

In Figure 5.1, the general concept of the system is presented. From a user's perspective, the system can be seen as a "black box" that takes the audio signal from the diver's microphone and outputs a version of the signal with the unwanted parts removed or attenuated. In this project, it is assumed that the input and output are digital audio signals. In a real-time implementation, an analog-to-digital and a digital-to-analog converter would likely be needed at the input and the output, respectively. However, a real-time solution is not within the scope of this project.



Figure 5.1: Diagram of the general concept of the system.

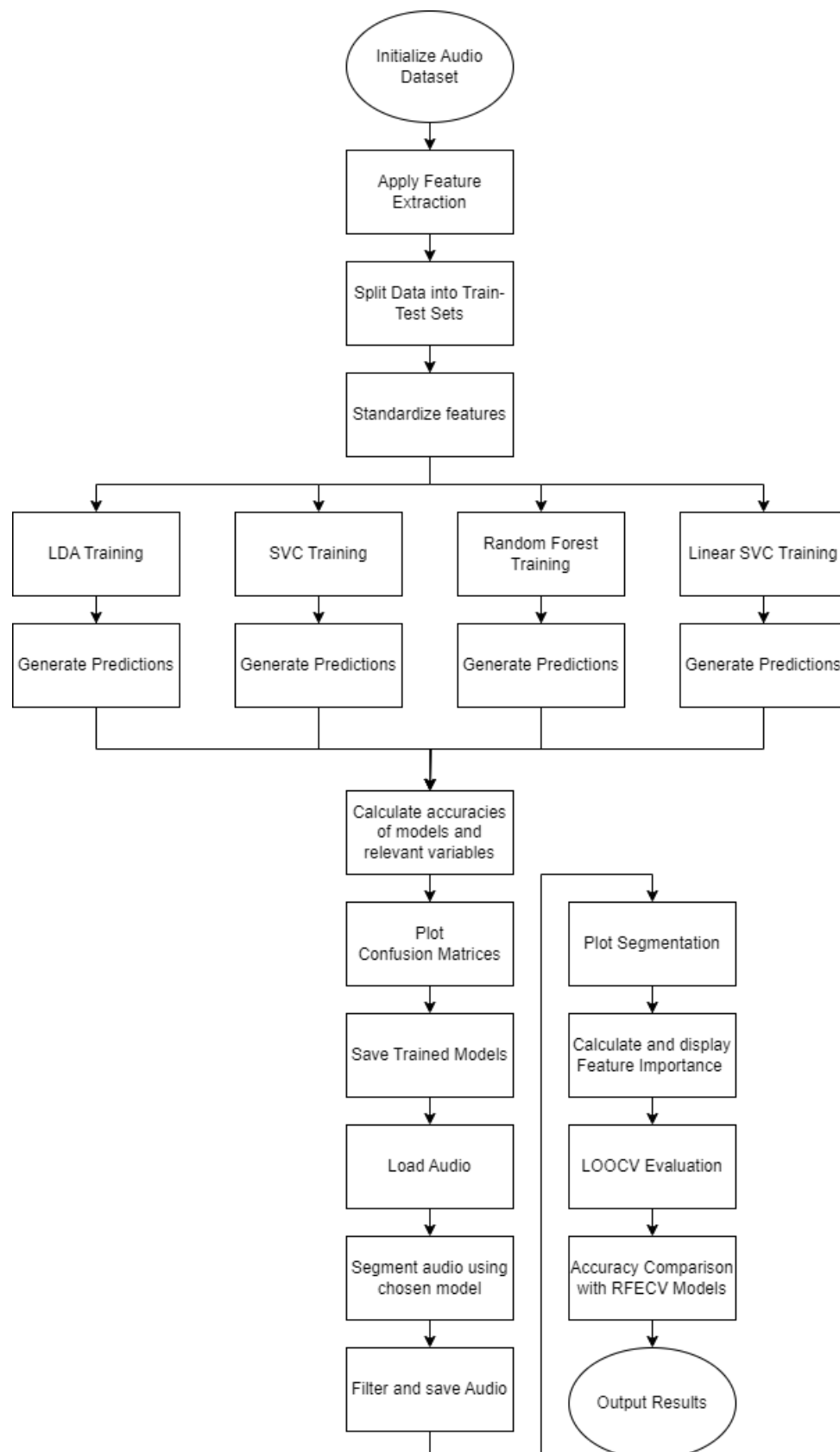
The purpose of the preprocessing stage is to make any calculations and changes to the signal that are necessary and/or improve performance before applying a machine learning classifier. It would likely be possible to simply calculate the features of some frame length and then use the machine learning predictor to decide how to attenuate that frame of audio.

The output of the "classification of segments" stage is the audio segments and their predicted classes. This can then be used in a post-processing stage. The post-processing then uses the information about the current category of the audio to choose differ-

ent kinds of filtering and different attenuation levels to apply to the audio signal. An important question is how the audio signal should be prepared before the post-processing/filtering stage.

The project focuses on the development of a machine learning based system. Regardless of how segmentation is done, the classifier will receive features extracted from some length of audio data, which means it should be possible to train a model to classify audio categories independently of this choice.

An overview of the code can be seen on Figure 5.2.

**Figure 5.2:** Flowchart of the code.

It shows the general structure of the main code. Each of these blocks represents a function and/or list of operations done sequentially. It shows the general flow of data through the system.

It is decided that while the previously made audio recordings from Nicoleta's project [1] are reused, the data will be preprocessed and annotated by the project group members.

The audio files were initially examined in the digital audio workstation program Audacity, as it was desired to get an understanding of the spectrum of different desired audio classes. Once an intuition of different audio classes had been obtained, the audio clips were segmented into the following categories.

- "Background": clips with no noise, or barely-audible noise from the show presenter.
- "Breathing": clips with the sound of the diver breathing.
- "Bubbles": clips with the sound of bubbles.
- "Speech": clips with the sound of the diver speaking.
- "Mix-Bubbles-Speech" ("MBS"): clips with the sound of the diver speaking, while bubbles can also be heard.

For ideal performance, the group would like to accurately identify each of these categories. These are the labels used for the given data.

With the data that Nicoleta gathered, the divers of each file were identified, and an index was created. This was done to make sure that each diver was represented in the data, and that if one file saw better segmentation performance, it was possible to identify the diver. The recordings were made of three divers: Kristina, Martin and Henrik. For each of these divers, 30 segments were made per label. This resulted in 450 segments in total. These segments were of differing sizes, since the contents of the segments were seen as more important than the length. This can, however, lead to some categories having more spectral data than others.

5.2 Development Practices

In the project, practices were followed to ensure that the written code was well-structured, while also avoiding issues like data leakage. To make sure that data leakage did not occur, the scikit practices and recommendations for proper preprocessing and for avoiding data leakage were followed [32].

In the code development process, GitHub was used for administrating and sharing code for the project. GitHub was used to keep order between "new" and "old" files in the project as the project progressed, but also to work with branches of code which were implementation-ready and in an experimental-phase. Both folders were kept under the same repository that was used for the project.

New code was initially written and tested in a branch "Experimental-code". After it was seen that code was functional, it was transferred to the "Master" branch.

In the Master branch, the used code was in the "New stuff" folder in the format of Jupyter notebook files (.ipynb). The Jupyter Notebook files used audio files from one of two folders. Two folders of audio data "Manually Segmented Data" and "Updated Test Training Data" could be used. "Manually Segmented Data" contained the diver audio data categorized as described in section 5.3, while "Updated Test Training Data" contained the diver audio data, where "Mix-Bubbles-Speech" and "Speech" audio segments were saved under the same label. As mentioned in 5.4, the latter combination was discarded due to lower obtained accuracies than with the former combination. Additionally, a folder "output_segments" was used to save audio files which had been attenuated or otherwise altered.

Project Code Practices

This subsection aims to explain some of the practices that were followed when writing code for the project. Namely, using Jupyter Notebook in Python to obtain code modularity and avoid redundant code interpretation.

Code Notebooks

The code was written in the format of Jupyter notebook files -.ipynb. Compared to a normal Python file (.py), Jupyter notebook files are different in the sense that they focus more on live code, equations, visualizations, and text for narrative purposes [37]. This proved useful, since the code iterations commonly saw experimentation and changes. Furthermore, it has the benefit of enabling code modularity, in which different blocks of functionality can exist within the same notebook.

The notebooks of the project were organized so that a code block was added at the top. In this block, the needed imports were made, and the project functions were defined. This can be considered a "setup" block. In the next block, the features were extracted with the defined functions, train and test sets were made, and the differ-

ent models were trained and saved. This generally worked really well, but for increased modularity, the latter code block may also be split into one block for feature extraction and one or multiple blocks for model training. Lastly, two code blocks were made where one was for saving trained models, and another was for loading the models which can be used for audio file segmentation, respectively.

Data Leakage

A typical cause of data leakage is training and test data subsets not being kept separately. The test data should never be used when training the model, since it would create an unrealistic idea of how well a model performs, while likely achieving worse results when implemented [32]. As a general rule, scikit recommends that the "fit()" method is never called on testing data in the pre-processing steps of data processing. Train and test data subsets should go through the same preprocessing transformation. Transformations should only be learned from training data.

Scikit proposes the following steps to avoid data leakage [32].

- Split data into train and test subsets before performing any preprocessing steps.
- Do not include test data when using "fit" and "fit_transform" methods - this can lead to overly optimistic results.
- Consider using the "pipeline" functionality to avoid data leakages.

It is important to note that the feature selection should only use the training data, and that data should be split into train and test subsets as the first thing.

5.3 Preprocessing

This section will focus on preprocessing the data, and related tasks. These will include: Feature extraction, The creation of training and test splits of data Standardization, and clustering.

Data Processing and Model Training

A folder named "Manually Segmented Data" is created, with subfolders for audio segments of the classes described in section 5.3. For every ".wav" file in the folders, audio features are extracted, using the main directory "Manually Segmented Data" as an argument to a function called "featureExtraction". Meanwhile, a second argument, "folders", which is a Python list, is passed to decide which types of audio clips should be used for the feature extraction. The function "featureExtraction" used the following features, through the "librosa" audio analysis library, to classify the audio segments[38]:

- Mel-frequency cepstral coefficients: 13 Mel-frequency cepstral coefficients (13 different features) are each averaged for each of the snippets.
- Zero crossing rate: the averages of the zero crossing rate for each audio snippet.
- Short Time energy: the averages of the short time energy for each audio snippet.
- Spectral contrast: the averages of the Spectral contrast for each audio snippet.
- Spectral Centroid: the averages of the spectral centroid for each audio snippet.
- Spectral Roll-Off: the averages of the Spectral roll-off for each audio snippet.
- Linear Prediction Cepstral Coefficients: The cepstral coefficients of the first 12 Linear Prediction Coefficients.

Lastly, "featureExtraction" returns "X", which is a feature vector, and "y", which is a vector of labels.

In the file, a function "featureExtraction" was defined. The function "featureExtraction" creates a feature extraction vector "X" based on the features of different labels. An argument "main_dir" describes the location of the files, while the second function argument "folders" determines the used labeling of used audio clips. Some of the code from "featureExtraction" is shown in Listing 5.1.

```

27 def featureExtraction(main_dir, folders,):
28     ##This creates a feature extraction vector. It Extracts features from
        the audio files using, MFCC, ZCR, STE, Spectral Contrast, Spectral
        Centroid, Spectral Rolloff
29     X = []
30     y = []
31     #The following "for-loop" iterates through folders, looks for .wav
        files, and constructs the full path to each of the .wav files.
        Feature extraction is applied to each file. Features are appended to
        the "X" list, while labels are appended to the "y" list.
32     for label, folder in enumerate(folders): #enumerate provides both
        the index (label) and the value (folder) for each iteration.
        Essentially, it assigns a unique label to each folder.
33         folder_path = os.path.join(main_dir, folder) #joins the main
        directory and the folder name to create the full path to the folder
34         for file_name in os.listdir(folder_path): #listdir lists all the
        files in the folder
35             if file_name.endswith('.wav'): #only looks at .wav files
36                 file_path = os.path.join(folder_path, file_name) #joins
        the folder path and file name to create the full path to the file
37
38                 # Load audio file (default is mono)
39                 audio, sr = librosa.load(file_path, sr=None)
40
41                 features = featureFunction(audio, sr)

```

```

42         X.append(features)
43         y.append(label)
44
45     return np.array(X), np.array(y) #returns the features and labels as
    numpy arrays

```

Listing 5.1: "featureExtraction" code

The Listing 5.1 captures the functionality of the feature extraction doing the following:

- Go through the folders that were passed.
- Look through .wav files.
- Extract their auditory features.
- Save them according to their label "y".

With "X" and "y" being defined through "featureExtraction", the feature extraction can be carried out. The function on line 41 in Listing 5.1 refers to a function called "featureFunction", which stores all feature extraction algorithms.

Training and Test Data

With "X" and "y" denoting the feature vector and set of labels, respectively, training and test-sets are made by splitting the original data sets, where 75% of the original data is used for training and 25% is used for testing. The feature training set is fit and transformed to a standard scaler to make the data normally distributed. The test data is standardized through centering and scaling. The pre-processed data is used for training a variety of models, which will be described in 5.4.

The "main_dir" may be defined as e.g. "Manually Segmented Data" if the original segmented data should be used. "Folders" may then be defined as ["Background", "Breathing", "Bubbles", "Speech"], if the "Mix-Bubbles-Speech" category is to be omitted. Having defined the directory and folders, the features may be extracted as shown in Listing 5.2, where it is also shown how the data is split into training and test data, and how the data is standardized.

```

1 #Point to the folders
2 main_dir = "Updated Test Training Data"
3
4 #Define folders of audio data to be used
5 folders = ["Background", "Breathing", "Bubbles", "Speech"]
6
7 #Obtain feature vector and labels according to directory and folders
8 X, y = featureExtraction(main_dir, folders)
9
10 #Obtain training and testing data

```

```

11 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=1/4,
12           random_state=42, stratify=y)
13 #Preprocessing - standardize features so the data is ready for training
14   and evaluation
15 #Standardize features: remove mean and scale to unit variance
16 scaler = StandardScaler()
17 #Fit the scaler on the training data and transforms it
18 X_train = scaler.fit_transform(X_train)
19
20 #Transforms the test data, with the same scaler fitted on the training
21   data
22 X_test = scaler.transform(X_test)

```

Listing 5.2: Code for Preprocessing, and splitting data into training and test set

With `X_test`, `X_train`, `Y_test` and `Y_train` defined, various functions can be called to train machine learning models.

Standard Normalization

The `StandardScaler()` class from the `scikit-learn` library applies standard normalization to data. It does this by subtracting the mean value of the data from each data point and dividing by the standard deviation. This effectively converts the mean and the standard deviation of the data set to become 0 and 1, respectively [39]. In this way, the standard score of a data point is calculated as:

$$z = \frac{x - \mu}{\sigma} \quad (5.1)$$

where: μ = the average of the dataset
 σ = the standard deviation of the dataset

The function `fit_transform(X)` both fits to the data `X` (calculates the mean and standard deviation of the data) and then transforms the data points using Equation 5.1. The function `transform(X)` simply standardizes the data if the mean and the standard deviation have already been calculated previously, also using Equation 5.1 [39].

Clustering

As part of the data preparation for training supervised models, clustering has been experimented with to get an idea of how many classes the features naturally can be put into.

In this way, clustering can give an understanding of how many different classes are needed to be trained.

Different kinds of clustering have been experimented with, namely "Affinity Propagation", "DBSCAN" and "Mean Shift". Of these, Mean Shift Clustering proved to be

most useful.

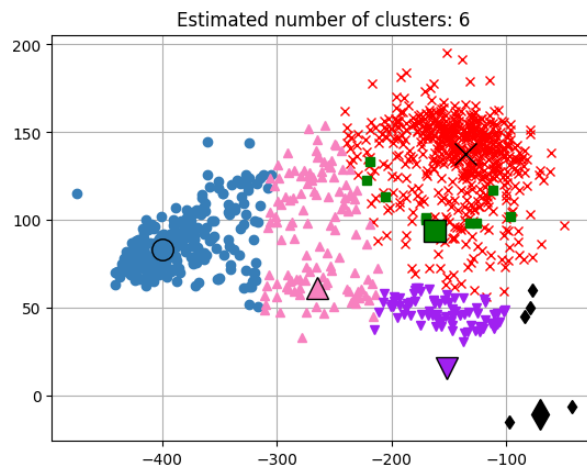


Figure 5.3: Mean Shift Clustering done on test data

In Figure 5.3, the result of applying mean shift clustering to the manually segmented test data can be seen. From this, 6 clusters are obtained, 5 of which appear to be mostly separable. The green cluster seems insignificant compared to the other clusters, as it consists of relatively few elements and is not properly separable from the red cluster. From this, it can be seen that there is merit to choosing less than 4 or 5 classes, depending on whether "MBS" is used or not.

The test data was already manually segmented based on auditory categories subjectively defined by the group. Based on this visualization of the clustering, it seems that these choices of categories can indeed result in somewhat separable groups of features.

5.4 Model Training

This section outlines the model training, and how they will be used to make predictions on test data. In section 5.3, a wide range of models will be trained, and their obtained results will be discussed.

With "X_test", "X_train", "Y_test", and "Y_train" defined, various functions can be called to train machine learning models. The code also defines functions to train machine learning models for LDA, SVC, Linear Support Vector Classification, Random Forest model, and Random Forest model with Recursive feature elimination. To maintain brevity, the specific code for all of these functions will not be written out. But in general, the functions all take "X_train" and "Y_train" as argument, with some of the functions taking more arguments. The functions return a set of values. E.g., the "ldaTraining" function takes "X_test", "X_train" and "y_train" as arguments, and returns trained LDA X-data, predicted y-values, and the trained LDA model. To

make an LDA-classifier, obtained predicted y-values and X-training data, the function may be called as shown in Listing 5.3

```
25 X_train_LDA, y_pred_LDA, classifier_LDA = ldaTraining(X_test,X_train,
    y_train)
```

Listing 5.3: Code for training data and obtaining model with LDA

LDA in this case refers to a Linear SVC model, with LDA applied to it. While the code in Listing 5.3 is specific for the LDA model, it shows the general way that a trained model, and predicted labels, are obtained by calling one of the functions. More examples may be seen by viewing the project code. Once "y_pred" is obtained, and "y_test" is obtained from the train_test_split function, it is possible to obtain the accuracies of the functions. To obtain the function accuracies, the function "printAccuracies" is called, which prints the accuracy score between "y_test" and "y_pred", the mean of the 10-fold cross-validation score, the standard deviation of the 10-fold cross-validation score, as well as the classification report - which contain the main classification metrics of the data, namely the recall, precision and F1-score.

Lastly, a few other functions were also created. A function "plotSegmentation" plots the segmentation of an audio clip based on labeled predictions of classes. Another function "Segmentation_using_predict" creates the segments to be plotted by "plotSegmentation". The function "plotConfusionMatrix" plots the confusion matrix of y_test and y_pred, and lastly, a function "plotDecisionBoundaries" plots decision boundaries for functions that group data into classes. These will not be explained in more depth here, but can be seen by viewing the code.

Comparison of Initial Models

In the initial work with machine learning, a few models and datasets were worked with to get an initial idea of the performance of different relevant models, and to get an idea of which performance was possible with different datasets. Initially, models of Support Vector Classification with Linear Discriminant Analysis, SVC, Linear Support Vector Classification, and a Random Forest model were trained, and their accuracies were compared against sets of test data. Testing the four models, the accuracies in Table 5.1 are obtained.

Model	Accuracy (%)
SVC	90.26
Linear SVC	89.38
SVC with LDA	79.64
Random Forest	92.92

Table 5.1: Overall accuracies obtained for four different models in section 5.3.

The accuracies of Table 5.1 are obtained using all five types of audio mentioned in section 5.3. A Decision Tree implementation, an RFE solution using a Random Forest model, and a Backward Feature Elimination using a k-nearest neighbor algorithm

were implemented. This time, the "MBS" category was not used, since it was hypothesized that excluding the category could lead to a higher accuracy. The rationale was that since "MBS" contains spectral data from both speech and bubbles, the model would more often misclassify speech or bubbles as this in-between state that "MBS" could be considered. With these models, the accuracies in Table 5.2 were obtained.

Model	Accuracy (%)
Decision Tree	96.29
RFE (RF)	98.66
BFE (KNN)	96.46

Table 5.2: Accuracies obtained for three different models with four audio labels

Since the results in Table 5.2 were achieved when labeling with one less parameter than the models of Table 5.1, it could not be concluded if it was due to the algorithms working better, or if it was due to the use of fewer categories of features. However, the results of Table 5.2 made it evident that the results of Table 5.1, coincidentally obtained with five labels, were too inaccurate. Thus, the test was re-done without the "MBS" label, obtaining the accuracies shown in Table 5.3.

Model	Accuracy (%)
SVC	98.88
Linear SVC	100
SVC with LDA	82.22
Random Forest	98.88

Table 5.3: Table: accuracies obtained for four different models, with four labels.

Table 5.3 shows that when the "Mix-Bubbles-Speech" label was removed from the prediction process, the accuracy of every model improved. SVC with LDA saw the smallest improvement of 2.56%, while LSVC saw that biggest improvement at 10.62%. It is worth noting that the LSVC accuracy change from 89.38% to 100% can be considered very impressive, since it becomes increasingly more difficult to achieve a higher accuracy the closer the accuracy is to 100%.

To understand the accuracies of the models better, the confusion matrices of the initially trained models are examined. The confusion matrices for SVC with LDA and RF are shown in Figure 5.4a and Figure 5.4b, respectively. LDA and RF are chosen to be shown, since they had the lowest and highest classification accuracies, respectively.

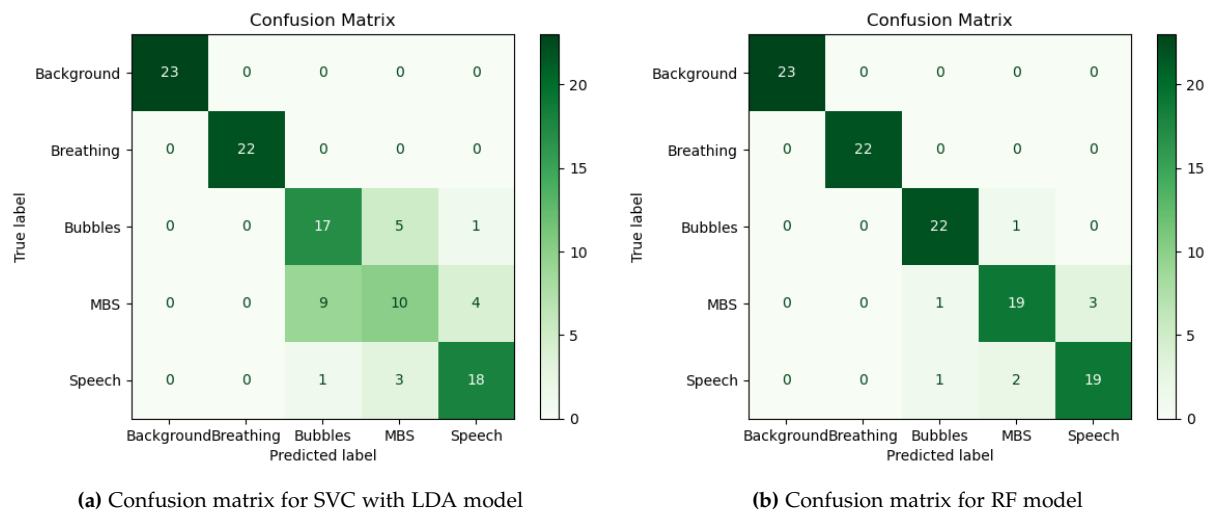


Figure 5.4: Confusion matrices for SVC with LDA model (Figure 5.4a) and RF model (Figure 5.4b) with five labelled audio categories.

Figure 5.4 is used to understand misclassification patterns. Figure 5.4a shows that the SVC with LDA has a perfect accuracy for "Background" and "Breathing", but struggles with "Bubbles", "MBS" and "Speech". "Bubbles" sees 17 TPs, but also 10 FPs and 6 FNs, being misclassified as "MBS" and "Speech". Likewise, "Speech" has 18 TPs, 5 FPs, and 4 FNs, with misclassifications often seen as "MBS" or "Bubbles". It is paramount that FNs in speech are avoided as this will interfere with the speech signal sent out to the audience.

The confusion matrix of the RF model in Figure 5.4b shows better results, maintaining a 100% TP-rate for "Background" and "Breathing". For "Bubbles" only 1 FN is seen, where it is identified as "MBS", and there are only 2 FPs where "Bubbles" is the prediction of "MBS" and "Speech" samples. "Speech" has a higher TP-rate of 19, generally seeing fewer misclassifications, although there is still some misclassifications where "Speech", "MBS" and "Bubbles" are confused for each other at varying rates.

Both Figure 5.4a and Figure 5.4b show that there is a noteworthy level of confusion between the categories of "Bubbles" and "MBS", as well as between "Speech" and "MBS". While Figure 5.4b generally shows fewer confusions, the misclassifications may still prove undesirable if a machine learning model is used on an audio clip and undesired audio elements cannot be muted. The inaccuracies may arise due to the fact that "MBS" contains elements of both "Bubbles" and "Speech". This means that the classes may be misclassified easily. The effect of removing the "MBS" label can be seen in the confusion matrices Figure 5.5a and Figure 5.5b, which are the confusion matrices of the SVC with LDA and Linear Support Vector Classification models with four labels.

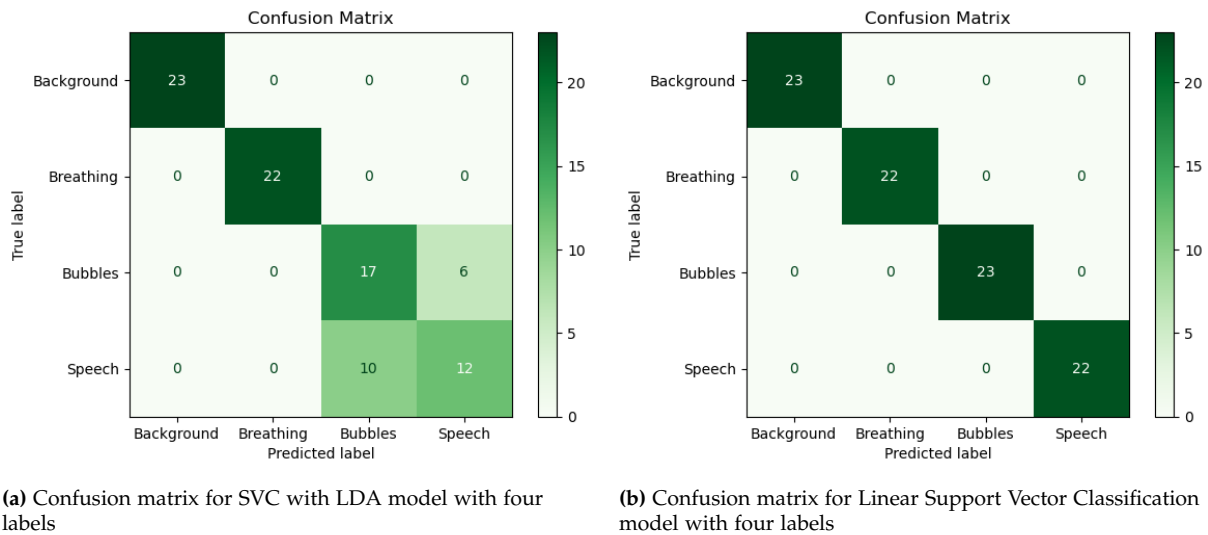


Figure 5.5: Confusion matrices for LDA model (Figure 5.4a) and RF model (Figure 5.5b) with four labelled audio categories.

Figure 5.5a and Figure 5.5b may be compared with Figure 5.4a and Figure 5.4b. For SVC with LDA, comparatively poor performance is still achieved, with a classification accuracy of 81.41%. It is seen that there are still many cases of "Bubbles" being classified as "Speech", and "Speech" being classified as "Bubbles". However, for the RF model, a labeling accuracy of 100% was achieved. This indicates that while some models (SVC with LDA) may not see any improvement, the removal of "MBS" can cause a large improvement for other models. As evident by the LSVC confusion matrix in Figure 5.5b, which has zero misclassifications.

It should be noted that the confusion matrices in this subsection reflect a relatively low number of results, due to the low number of segments used. The segments had different lengths, some being longer than 1 s. This resulted in fewer classifications in general. It could be hypothesized that using more segments for the training and classification process might improve the model robustness, but the used clips were labeled to match their audio type, making them reliable for predictions. While a larger dataset may be advantageous for the training and predictions, the current data is considered sufficient to give an accurate view of the model precision in spite of the smaller sample size.

An updated test/training set was made in which the audio clips of "MBS" and "Speech" were combined into one label "Speech". This was experimented with since it was thought that it might improve the accuracy in detecting speech. However, the achieved accuracies were lower than the ones in Table 5.3, so it was discarded.

The takeaways of this subsection are that when working with 4 audio categories instead of 5, a higher classification accuracy score is seen. It is seen that the SVC with LDA model sees an insignificant accuracy improvement when using fewer labeled categories, indicating that it might not perform well for the classification task. In

chapter 6, the final models will be tested, and their results will be presented.

5.5 Model Application

Once the model has been trained, the model can be applied to audio files, with an expected performance comparable to the scores calculated in 5.4. This labeled audio can then be processed to attenuate the unwanted audio classes. In order to achieve this functionality, functions have been developed, the main ones being "Segmentation_using_predict()", and "filterAndSaveAudio()".

Listing 5.4 shows how these functions are applied:

```

25 #Takes the audio file and connects it to the audio path, leading into
    the rest of the code
26 audio_path = "Audio file name/path"
27 save_path = "" # Path to save the output audio file
28 Modelfolder = "Trained models"
29
30 #Example of loading a trained model from the files in the folder
31 svcModel = load_model(Modelfolder, 'svcModel.pkl')
32 scaler = load_model(Modelfolder, 'scaler.pkl')
33 # Load the audio to get the sample rate
34 audio, sr = librosa.load(audio_path, sr=None)
35
36 # Applies the ML model to the audio file and segments it based on the
    model's predictions
37 segments = Segmentation_using_predict(audio_path, linSVC, scaler,
    window_size_seconds=0.1, hop_length_seconds=0.1, min_segment_seconds
    =0.1)
38
39 # Plots the audio waveform with the adaptive segmentation applied
40 plotSegmentation(audio_path, segments, folders, 60, 'min.')
```

Listing 5.4: Code for Applying a machine learning model and attenuating the audio

```

1 #Filters and saves the audio file based on the segmentation
2 filterAndSaveAudio(audio_path, segments, save_path, attack_duration=0.01,
    bubbles = 0.1, Breathing=0.05, Background=0.05)
```

Listing 5.5: Code for filtering and saving the audio

Listing 5.4 shows an excerpt of code from "finishedProgram.ipynb" in which a code block has been made for segmenting the audio file and plotting the audio waveform after segmentation has been applied to it. In Listing 5.4, it is shown how the path to the audio file is stored in "audio_path", while the model folder is loaded as "Trained models", which is the location in which the trained models have been saved. With this being defined, models can be loaded through the function "load_model()", which is a function that loads a specified model in a ".pkl" format, from a specified folder. After defining a model, a scaler is also defined. From the librosa "load()" function, the audio and sample rate are defined, after which the segments can be found. An example of how this can be plotted is seen in Figure 5.6.

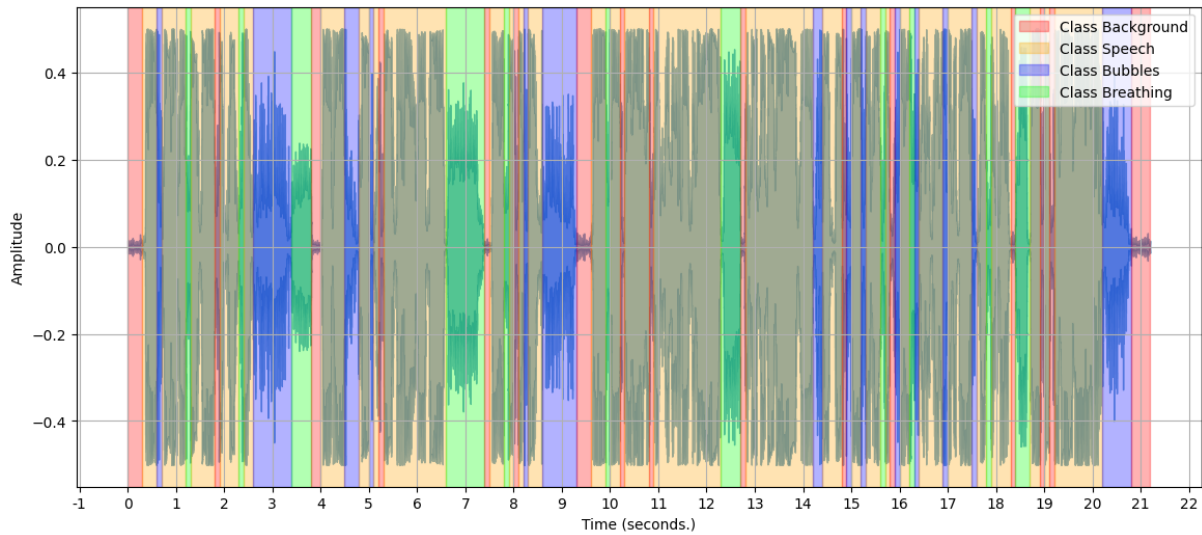


Figure 5.6: This is an example of the segmentation of diver audio using one of the trained models. Each part of the audio segment is given a color matching the class it has been predicted to match.

The "Segmentation_using_predict()" function loads the Linear Support Vector Classification model and the scaler, and uses them to classify audio segments. Here, a window size of 0.1 s is used, and the hop-length is also 0.1 s, meaning that the window is moved by 4410 samples for each prediction. When this creates the segments, the segmentation can be plotted through the function "plotSegmentation()", which loads a .wav file from a folder, loads the segmented data from the "Segmentation_using_predict()" function, applies the labels defined by the audio categories in "folders", and lastly configures the x-axis to be a timescale of "minutes".

Lastly, "filterAndSaveAudio()" filters and saves the audio based on the created labeled segments. This is shown in Listing 5.5. The "attack_duration" can be modified, and is an expression of the attack time - the delay between the time instance where a threshold is met, and when the inverted ASR envelope is afterward applied. "ASR" refers to the different sections of the envelope: "Attack", "Sustain", and "Release". The arguments and values for "bubbles", "Breathing", and "Background" are saved to a dictionary "attenuation_factors" in the function, which defines the attenuation factors for the categories "Bubbles", "Breathing", and "Background". In the classification of audio segments, this is used as such. The segments are iterated through in a for loop, where the class names are obtained from the labels of the folders of the audio categories. A variable "target_attenuation" is then modified according to the attenuation factors in the dictionary, such that according to the label of the segment, the desired attenuation factor is applied. With this, the envelope is applied to the part of the audio based on the length of the segment and the attenuation factor. Once all the segments have been iterated over, the attenuated audio is saved, finalizing the process of filtering and saving the audio. Note that "filterAndSaveAudio()" has its own listing, since it has its own code block in the code. To see the code of the aforementioned functions, see the notebook "finishedProgram.ipynb".

5.6 Model Optimization

The objective of this section is to explain how the models are optimized with regard to the used features. It will explain how ranking was carried out to decide which features should be used for the different models, since an optimal choice of features will lead to optimal model performance.

Feature Selection and Dimensionality Reduction

In the implementation of the models, versions using LDA and models using RFE have been developed. These utilize functions from the SciKit library. Specifically, the functions "LinearDiscriminantAnalysis()" and "RFECV()". As explained in section 3.3 and subsection 3.6, these can be used to reduce the size of the feature vector. This can, at times, not only reduce computation time, but also improve the system accuracy. This is due to not all features in the vector being useful for the given classification. In a worst case scenario, certain features can make a model less accurate if they catch a pattern in the signal that is irrelevant to the classification. This can confuse the model into thinking that this is relevant for the classification, increasing classification mistakes. Therefore, choosing the correct features is important for the overall performance. The number of features should be as low as possible, while providing enough information for the model to correctly identify the classes in all relevant scenarios.

From the initial results presented in 5.4, it was seen that SVC with LDA led to a significant reduction in the prediction accuracy as compared to both SVC and LSVC. Therefore, it is decided not to use SVC with LDA in the final testing. However, LSVC showed promising accuracies when four categories were used, so it will be used in the final testing. It was also seen that RF performed well, and that RFE performed well in the instance where it was used with RF. Therefore, RF will also be tested, and RFE will be tested for a RF model and an LSVC model.

The function "RandomForest_RFECV_pipeline" trains a random forest model using recursive feature elimination and also cross validation.

Then, the function "process_selected_features(rfecv, feature_names)" prints out the most important features that the recursive feature elimination algorithm has chosen and displays the importance scores of each of these features. These features and importance scores can be seen in Figure 5.7.

Feature Importance

Feature importance is a measurement of how much each element in a feature vector contributes to the overall accuracy of the model. As part of the the code, recursive feature elimination is applied and the feature importance is calculated for the chosen features. The calculated feature importance can be seen on Figure 5.7. Note that these calculations do not take correlation between features into account.

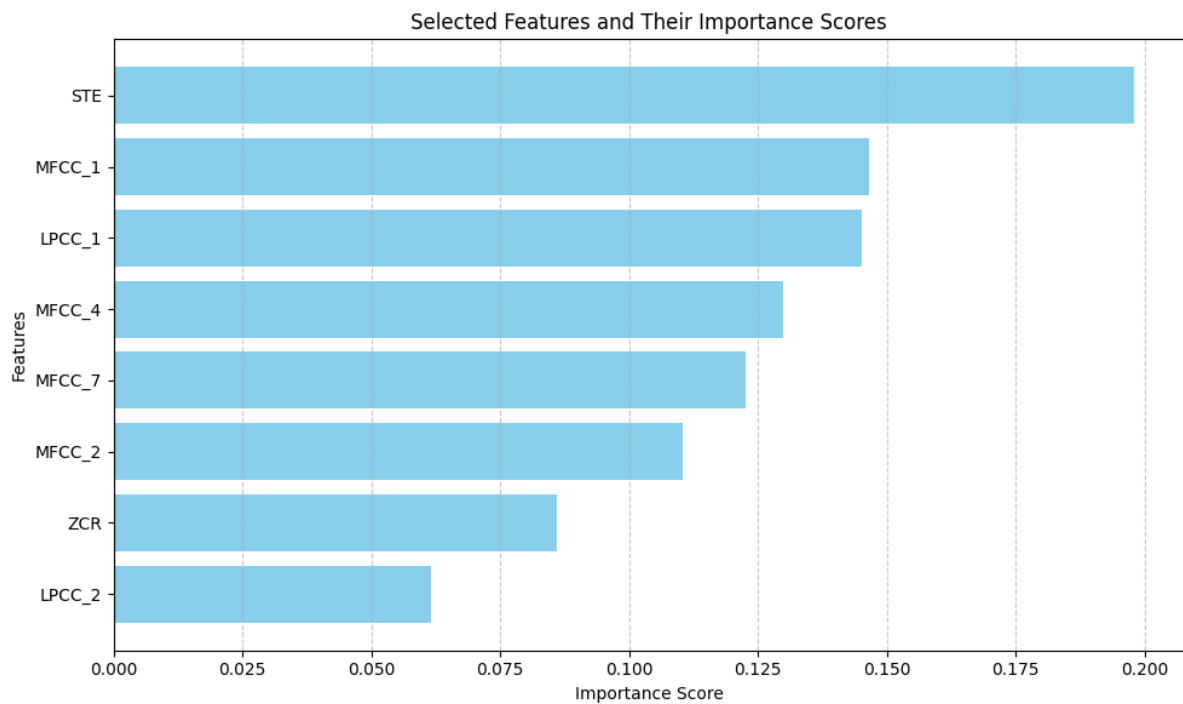


Figure 5.7: The importance of the selected features of the RFE for a random forest model.

In sci-kit learn, all tree-based models have a built-in method to get the importance of the features after the model is trained. The importance of a feature is related to how much it contributes to reducing the impurity. [40]

“If a feature is capable of creating pure splits in the tree and contributes to a cleaner partitioning of the data, that feature will have a higher importance.” [40]

When growing a decision tree, the dataset is recursively being split based on the features to create a tree-like structure. At each split node, impurity is measured [40]. Two common criteria to measure impurity are Gini impurity and Entropy / information gain [40]. Information gain was briefly mentioned in section 3.2.

After the tree has been constructed, the feature importance of each feature is determined based on the number of times it was used to split nodes and the reduction in impurity it achieved [40].

Choice of Model

The choice of ML-model affects the precision, the execution time, and the accuracy of the prediction.

This may be worth keeping in mind if the model should be applied in real-time, or if the given system is resource limited in some capacity (low processing power, battery powered, etc.) Of the models currently being discussed, two stand out: The Random Forest model, and the Support Vector Machine, specifically Linear SVC. While

both provide comparable performance, Linear SVC tends to have a faster prediction time. Furthermore, LDA and/or feature selection can be applied to these models, which, in theory, should reduce the amount of data being processed at any given time (section 3.3, section 3.6). As such, if the computation time were the most important factor when comparing these two, the linear SVC model would be the obvious choice. However, if accuracy is the greatest factor, then the preliminary results suggest that Random Forest model may have an advantage. In most applications, a balance between these two factors must be struck in order to find the ideal model for the given system. It is worth mentioning that ensemble models are also possible, utilizing multiple models. For the purposes of this project, these were not explored.

Classes

The number of classes tends to also affect the system's performance. As such, choosing the right amount of classes for the given system is important. Previously, the number of classes used in the system was 5, but now it has been reduced to 4, which was discussed in section 5.4. This increases the system's accuracy significantly. This is done partially based on the results found in subsection 5.3. The results suggested that there should not be used more than 5 classes. Having too few classes can make it difficult for the model to focus on classifying correctly due to not having enough classes to ensure separability. Too many classes can increase the complexity and can be too detailed for a given application, causing overfitting.

6: Test

Tests have been developed to see if the system fulfills the requirements set in chapter 4. The requirements will be verified with the following tests:

- Performance Test
- Audio Prediction and Attenuation Test

Different algorithms and techniques for machine learning will be tested. Most of the tests for this system will be performed in the same way, but they will be repeated with different models, testing their performance. The following models were chosen to be tested: Linear SVC, RF, RF with RFE, and Linear SVC with RFE.

6.1 Model Performance Test

This test focuses on training the models, and calculating the performance parameters for each model. This is done to see how well the system recognizes speech signals. Based on this, the models will be evaluated, and it is chosen which models are worth doing further tests on.

Test Setup

The following items are required to be able to conduct this test.

- A PC
- The finished program code
- The test/training dataset

The test is done on the following models that can be found in the "finishedProgram.ipynb" program.

- Linear SVC (linSVC)
- Linear SVC with RFE (rfecvLinSVC)
- Random Forest (rfModel)
- Random Forest with RFE (rfecvRF)

The program prints a performance report that contains performance metrics for the various classes. Classification "3" corresponds to the "speech" category, and these

values are used to evaluate the performance.

The manually segmented data is used, with 75% for training and 25% for testing. The chosen models are trained using the training data. The test set is used to calculate the performance of the chosen models.

For the models to pass the requirements, the following criteria from chapter 4 should be met:

- The measured accuracy matches or exceeds 0.9768
- The measured precision for speech matches or exceeds 0.97
- The measured recall for speech matches or exceeds 0.97
- The measured F1-score for speech matches or exceeds 0.97

Test Results

The performance test results will be presented, starting with Linear SVC in Table 6.1.

Class	Precision	Recall	F1-Score	Support
0 (background)	1.00	0.96	0.98	23
1 (breathing)	1.00	1.00	1.00	22
2 (bubbles)	0.96	1.00	0.98	23
3 (speech)	1.00	1.00	1.00	22
Accuracy	0.99			90
Macro Avg.	0.99	0.99	0.99	90
Weighted Avg.	0.99	0.99	0.99	90

Table 6.1: Linear SVC Classification Report

The scores in Table 6.1 are close to ideal, as the model has perfect scores for the speech class. However, it struggles with the background and bubbles, possibly confusing the two. While not ideal, the scores are still within reason, and therefore, the classification of these should be accurate for the most part.

Class	Precision	Recall	F1-Score	Support
0 (background)	1.00	1.00	1.00	23
1 (breathing)	1.00	1.00	1.00	22
2 (bubbles)	0.96	1.00	0.98	23
3 (speech)	1.00	0.95	0.98	22
Accuracy	0.99			90
Macro Avg	0.99	0.99	0.99	90
Weighted Avg	0.99	0.99	0.99	90

Table 6.2: Random forest Classification Report

The scores of the RF model in Table 6.2 are more troublesome. While it perfectly classifies both background and breathing, it seems to struggle with bubbles and speech. This is not ideal, as classifying speech correctly is the main focus of the system.

Class	Precision	Recall	F1-Score	Support
0 (background)	1.00	1.00	1.00	23
1 (breathing)	1.00	1.00	1.00	22
2 (bubbles)	0.95	0.91	0.93	23
3 (speech)	0.91	0.95	0.93	22
Accuracy	0.97			90
Macro Avg	0.97	0.97	0.97	90
Weighted Avg	0.97	0.97	0.97	90

Table 6.3: Random forest with RFECV classification Report

The scores of the RF with RFECV model in Table 6.3 follow a similar pattern as the scores in Table 6.2. However, the scores for speech and bubbles seem to have worsened. This may be due to relevant features having been removed, making the model predict faster at the cost of the model's accuracy.

Class	Precision	Recall	F1-Score	Support
0 (background)	1.00	1.00	1.00	23
1 (breathing)	1.00	1.00	1.00	22
2 (bubbles)	1.00	1.00	1.00	23
3 (speech)	1.00	1.00	1.00	22
Accuracy	1.00			90
Macro Avg	1.00	1.00	1.00	90
Weighted Avg	1.00	1.00	1.00	90

Table 6.4: Linear SVC with RFECV classification Report

The scores of the Linear SVC with RFECV model in Table 6.4 are perfect. All classes are correctly identified with no mistakes. While this might seem ideal, it can point to

an underlying problem of the model overfitting to the data. Given these scores, this is a possibility that should be explored to ensure the generalizability of the model is not compromised.

Based on these results, Table 6.5 was constructed. Here it can be concluded which models are worth considering for further testing.

Model	Accuracy ≥ 0.9768	Speech Precision ≥ 0.97	Speech re- call ≥ 0.97	Speech F1- score ≥ 0.97	Req. Met / Not Met
Linear SVC	0.99	1.00	1.00	1.00	Met
Random Forest	0.99	1.00	0.95	0.98	Not Met
Linear SVC with RFECV	1.00	1.00	1.00	1.00	Met
Random Forest with RFECV	0.97	0.91	0.95	0.93	Not Met

Table 6.5: Test results from the tested models

As can be seen in Table 6.5, while the RF models do perform quite well, they do not meet the set requirements for the system. Both iterations of Linear SVC pass, where the RFECV iteration has perfect scores across the board. While impressive, this could indicate overfitting, and therefore, a model that is bad at generalizing. This will be explored further in the following tests.

6.2 Audio Prediction and Attenuation Test

This test builds upon the test results in section 6.1. It is used to assess how well the trained model predicts, and how the system attenuates unknown data. The "unknown data" in this context refers to data that the model or the system has not previously seen in the training or testing.

Test Setup

For this test, the following materials are required.

- A PC
- A diver mic audio ".wav"-file not used in the training or testing of the model
- The chosen trained model

For this test, audio files with a length of 3 min from the three different divers have been chosen. These audio files are from parts of the diver recordings that have not been used as test or training data. These files all contain a mix of speech, background, bubbles, and breathing.

The following steps are done to conduct the test:

1. An audio file for each diver is chosen.
2. These files are passed through the prediction and attenuation algorithms.
3. The attenuated output audio file is saved and compared to the original audio file in the program Audacity.
4. In Audacity, the waveforms of the two files are compared to find intervals of potential speech attenuation

The test is passed if the system does not attenuate any speech.

Test Results

In Figure 6.1, a snippet of an audio file containing a recording of the diver Kristina can be seen. This snippet shows a comparison between the original file at the top, and the processed file at the bottom.

The red vertical lines indicate a time interval in which speech has been incorrectly removed.



Figure 6.1: Figure showing attenuation of a full word by the model.

As can be seen in Figure 6.1, the word "fisk" is being attenuated out of the file, causing information loss. However, partial incorrect attenuation of words is sometimes also seen. This can be seen in Figure 6.2. This figure follows the same structure as Figure 6.1.

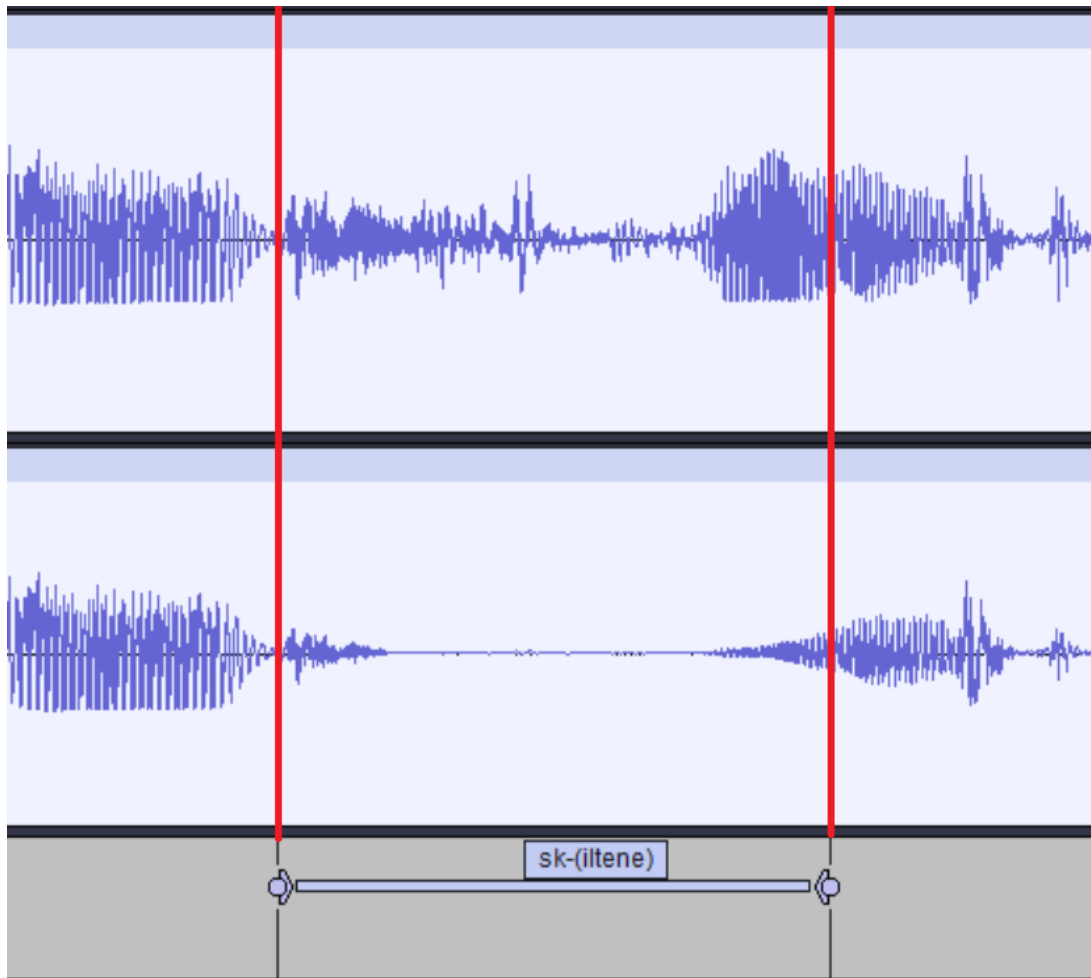


Figure 6.2: Figure showing partial attenuation of words. The letters outside the parentheses represent the attenuated/missing parts of the word, and the letters inside the parentheses indicate the rest of the word (which is audible).

From Figure 6.2, it is apparent that the "sk" start of the word "skiltene" is attenuated away. While some contextual clues might help fill in the gaps in the sentences, it is undeniably still information loss. If this audio were played through a speaker system in a reverberating room, such as the one at Nordsøen, it is quite possible that the contextual clues might not be sufficient.

These characteristics were present in all inspected files, and as such show a model that is still not trained to perfectly separate speech and non-speech. The specific parts lost seem to suggest that the model has difficulty distinguishing unvoiced sounds. This could point to a revised model being necessary.

As such, it is concluded through this test that R5 from Table 4.1 has not been met.

7: Discussion

In this chapter, the test results and the general state of the project are reviewed. What was achieved and what can be concluded is discussed, and thoughts about the importance of the requirements are made. Ideas are presented about the further work required to obtain the functionality that the system was meant to achieve. Lastly, the chapter discusses the system limitations that were discovered during the project and what could be achieved with further work.

7.1 What was Achieved

For the purpose of solving the problem statement, a program has been developed. This was developed based on the theory in chapter 3, and the problem analysis in chapter 2. Based on this, the requirements for the system in Table 7.1 were chosen.

Requirement	Description	Traceability
R1	The classification accuracy for the system should be at or exceed 0.9768	[1]
R2	The precision for the speech detection should be at or exceed 0.97	[1]
R3	The recall for the speech detection should be at or exceed 0.97	[1]
R4	The F1-score for speech detection should be at or exceed 0.97	[1]
R5	The system must not attenuate speech parts of the audio signal.	

Table 7.1: Requirements.

From the requirements in Table 7.1, requirements R1 through R4 were all met to some extent. More specifically, the following can be said about the individual requirements and the extent to which they were satisfied:

- R1: From the summary presented in Table 6.5, it can be seen that the models LSVC, RF, and LSVC with RFECV satisfy the accuracy requirement, while RF with RFECV fails to fulfill the requirement.
- R2: LSVC, RF and LSVC with RFECV satisfy the speech precision requirement, but RF with RFECV does not satisfy it.
- R3: The desired speech detection recall is only met by LSVC and LSVC with

RFECV, while the RF and RF with RFECV models did not satisfy the requirement.

- R4: an f1-score ≥ 0.97 was achieved by LSVC, RF and LSVC with RFECV, but not by RF with RFECV.
- R5: The final model used for the testing was a LSVC model. When the model was applied to see if it could attenuate the undesired parts of the output signal, information loss was seen both from full and partial attenuation of words. Thus, R5 was not satisfied.

In the project, it was possible to develop two models that satisfied the requirements in R1, R2, R3, and R4. The final RF model did not satisfy the requirements, but came close to satisfying the speech recall requirement. The RF with RFECV model did not satisfy the requirements, even though initial work showed that an RF model with RFE might be able to achieve high classification accuracies. With more time, further work could be carried out to see why the final RF model with RFE and CV performed worse. The last test also showed that the requirement R5 was not achieved. This may be due to the requirement being quite strict. Overfitting could also be a concern. It is also possible that the model would perform better if given more test data. This would require further testing to find out, with the issue of attenuating unvoiced sounds being left open for further investigation.

In general, the work of the project achieved to make two machine learning classification models that satisfied most of the requirements of the project. However, it was not possible to meet all of the requirements by any of the models. The RF model failed, which can be attributed to the speech recall. A lower recall means that more positive cases are identified as negatives, meaning that a lower recall for "speech" means more FNs have been seen. That is, since the model misses true positives, it might miss out on "speech" instances during classification. The RF with RFECV model not only makes several FPs for the "speech" class, but also has the same recall as the RF model, meaning that its performance is not sufficient. Furthermore, its F1-score for speech, is too low.

7.2 System Limitations

While the system meets most expectations, there are still limitations to the system which are worth considering.

Data Driven Training and Testing

It is generally accepted that the performance of a machine learning model will never be better than the training data that it is trained on. The training data of a machine learning model may be made to be similar in its early phase to limit its variability, since the model is unlikely to generalize well early in its training. However, training data may also be diversified to avoid the issue of overfitting, since a larger data set can result in better fits for higher order models. In general, it may be said that the issue of training a machine learning model is the following: if a model is underfit to training data, it means that there is room to improve the model. If a model is overfit, it means that it is fit to the noise of the data. This means that it is precise for making classifications of test data/data points, but does not perform well at capturing general data tendencies. It is important to note that the training data will contain variations, and it is important to question whether the variation of the training data will be similar to the variations seen in new test data. It is also important to consider how well the created machine learning model can generalize - that is, how well it performs on unseen data. Furthermore, it is important to consider how well the model can handle changes in the characteristics of the data. Hypothetically, it could be an issue if the characteristics of the test data were different from the ones of the training data - a "domain change". Currently, this is not an issue in this project. But if the divers at Nordsøen Oceanarium were to be changed, it could pose a problem to the accuracy of a system, assuming that it does not generalize well.

Considering this project, it is evident that the trained machine learning model performed decently on the test data. It should be considered that training data was made to reflect the audio properties of the different divers at Nordsøen Oceanarium. This did not limit the variability of the data, but helped diversify the data. This was useful since the test data could also contain audio from several divers. Since the model was tested on test data, the variations of the test data should be similar to the training data. No significant changes in characteristics occurred, and the variations in the test data were consistent with those of the training data.

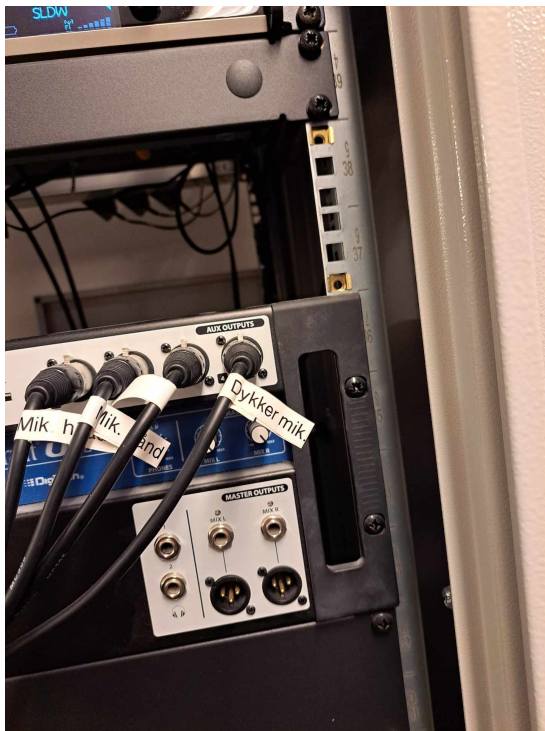
The test data suggests that the model performs decently. However, it is yet to be explored if training the model on data from a single diver can provide more accurate classification results. Given more time, future work could investigate the model's adaptability to data from divers not in the training set, thus further evaluating its robustness on unseen data.

7.3 Further Work

While the system in many ways achieves what was meant to be achieved, system improvements could still be made. More efforts toward optimizing and implementing a real-time version could be done. On-site implementation and development of the necessary hardware for such an implementation could also be worked on going forward.

Real-Time Implementation

This project worked with pre-recorded audio files, aiming to see if a solution could be constructed which minimizes the presence of unwanted audio elements, reducing the amount of distracting elements when listening to the diver. However, this work only addressed pre-recorded files. This subsection will explore the possibilities of implementing a filtering solution in real-time, on the basis of the technical setup at Nordsøen Oceanarium. A real-time audio filtering implementation may be possible on the equipment used at Nordsøen Oceanarium. In order to do so, the system it has to interact with must be taken into consideration.



(a) Diver microphone cable, "Dykker mik", going into a Soundcraft Ui16 Digital Mixer



(b) The "London Blu 100" signal processor used in the audio setup

Figure 7.1: The diver and DSP setup at Nordsøen Oceanarium

Figure 7.1a shows the diver microphone cable, labelled as "Dykker mik", going into a Soundcraft Ui16 digital audio mixing console. After the microphone signals of the diver enter the Soundcraft Ui16, they go into a signal processor before being trans-

mitted through the speakers for the show. Figure 7.1b shows the "London Blu 100" signal processor used in the audio setup at Nordsøen Oceanarium. This is the last part of the audio processing equipment chain before the audio is played through speakers in the Oceanarium. To consider a real-time implementation, this equipment should be explored further to understand the possibilities.

The challenge of the system is to insert a machine learning model into the signal processing chain such that it can process audio in real-time. Considering the manuals of the Ui16 and the London Blu 100, the Blu 100 seems to be a more powerful tool for adjusting the audio. In the system, it is important that the machine learning model is placed such that the input into it matches the data it was trained on. Furthermore, while the current working machine learning model is trained on ".wav" files captured from the Ui16, and can do classifications of .wav files, a real-time system would likely deal with live signals. This is a challenge, since the live signals in the Ui16 will not immediately be available as ".wav" files. To solve this would include using buffers to store the input in and passing it along, instead of loading a complete file. Keep in mind, the buffer will cause a delay between input and output, proportional to the buffer size.

When exploring the Ui16, it seems that there are many potential solutions - but a practical solution should be found. The potential implementations can be speculated on based on documentation about the Ui16. The Ui16 has a USB audio interface and Ethernet ports, which can stream the audio over a local network [41]. Similarly to what Nicoleta did [1], live audio may be captured through the Ui16, and then used for classification. The Blu 100 has GPIO and RS-232 integration for third-party systems, meaning that a machine learning model may be put to run on an external device connected to the Blu 100 [42]. Another issue would be that when the audio enters the Blu 100, it has been through EQ-mixing on the Ui16, as shown in Figure 2.1. The output of the Ui16 will not necessarily match the training data, which could make the model less accurate. It may be more feasible to modify the signal before it enters the Ui16 using a hardware solution.

In any real-time digital signal processing, there will be latency added to the signal. The amount of latency will depend on the application. A balance is often needed to be struck to achieve ideal performance. The given use case gives a considerable amount of leeway in accordance to the acceptable amount of delay. Given that the mouth of the diver is not visible to the audience, a delay that would otherwise be distracting may be acceptable. It is also assumed that the audio is not fed back to the divers' headset, as it otherwise might affect their ability to speak, depending on the induced delay. The limiting factor will be how it affects the communication between presenter and the diver. Ideally, the change in their communication should be as little as possible, ensuring that it does not change the experience of the audience. Lastly, it is worth considering that the ML-model used will significantly affect the latency, with certain models not being well suited for real-time implementation. As such, prediction time will become a relevant metric for choosing the specific model

to use, while currently no requirements for this are set.

Correlation of Features

For this project, RFE was used for feature selection. RFE does not directly take the correlation of the features into account. In order to address the correlation in future work, "Exhaustive Feature Selection" could be used, as it takes all possible combinations of features, and thereby correlation, into account [24]. Otherwise, analysis of correlation of the features chosen could also be done.

Speech Classification Accuracy

While the initial model's test result in section 5.4 showed high performance metrics for the developed models, further testing showed that the model's speech prediction could still be improved. It is possible that with a larger training and test set, the system would have performed better on unseen diver audio. This should be considered if further work is done on this project. Furthermore, work could be carried out to see if the model has an issue of identifying unvoiced sounds, or if there was another source of the classification issues seen in the attenuation test results in section 6.2.

7.4 Worth Considering

The auditory experience at the facility should also be considered. The room where the audio is played is in front of a large aquarium, which means that a large flat glass surface is inherently part of the experience. As a result, the audio from the speaker system and any other audio source in the room, will be reflected around the room, affecting the experience. While this can be mitigated through the utilization of acoustic absorbers in the room, the room will inherently have a significant amount of reflective surfaces. Another consideration is the audience. The audience tends to be made up of a high percentage of children, which leads to the room usually being quite noisy, making it more challenging to hear what is being said by the diver and presenter. This could partially be mitigated by, within reason, increasing the volume of the audio being played. The environment can therefore, be a distracting element by nature, limiting how big of an improvement can be achieved without making significant changes to said environment.

Headset Interference

At the visit to Nordsøen Oceanarium, it was explained that the diver during the diver session would bring a "headset" consisting of 2 disc-shaped speakers. These can be either left floating in the water or placed on the head of the diver under the mask strap. The headset is used for communication between the diver and the presenter, and also plays back the diver's own voice. Because of the loudness of the "headset", the sound from it can be heard through the diver's microphone, which can end up

leading to misclassifications in the system as the background noise from the speakers contain speech. This could be mitigated by using a noise gate before the system. Assuming the threshold is set higher than the picked up speaker volume, then this should mute it, and there would be no reason for the model to classify it as speech.

Length of the Window

An important factor when applying digital signal processing is the window size that is used. This can have a significant effect on how the system reacts to the signal passed through it. Given the audio application, it is expected that using smaller windows could lead to the phonemes¹ being detected individually, while using larger windows could lead to voiced segments being classified as a whole. These phonemes would be in categories of "voiced", and "unvoiced". These phonemes can at times be as short as 10 ms[44]. This system change has however not been tested and would require further research to confirm.

At the hand-in of the project, it was seen that the attenuation of an unseen audio file did not fully match the accuracy achieved by a trained machine learning model. It can be hypothesized that this was due to the size of the window. The window had a length of 100 ms, which could cause misclassifications. Consider, e.g., the case in which a window covers 51 ms of "breathing", and 49 ms of "speech". In this case, the segment may be classified as "breathing", and attenuated, even though it means a loss of information from the diver. This example can also be considered for any other case where a class of audio that it is wished to be attenuated takes up a majority of the window, causing "Speech" to become attenuated. An approach may be explored to see if this issue can be avoided or mitigated.

Consider the option of implementing a hoplength between the windows, while the window length is still 100 ms. Reducing the hoplength would mean that the largest possible misclassification which can occur would be reduced. This may mitigate the issue of incorrect classifications. Generally, decreasing the size of the hoplength may mitigate the potential risk of muting speech due to the presence of other detected audio types in the window. However, this was left unexplored at the hand-in of the project.

¹"Phoneme, in linguistics, smallest unit of speech distinguishing one word (or word element) from another, as the element p in "tap," which separates that word from "tab,"[...]"[43]

8: Conclusion

The main focus of this project has been to analyze mixed audio signals of a diver in Nordsøen Oceanarium, aiming to classify the audio and dampen or entirely remove distracting elements from the signal.

This led to the final problem statement:

"How can a machine learning system be constructed to segment and classify the diver's audio signal?".

To answer this, the following topics were researched:

- Machine learning models
- Feature engineering
- Adaptive and non-adaptive segmentation
- Relevant machine learning performance metrics
- Relevant audio processing theory

Based on this, work was done to create a system that utilizes a machine learning model to classify audio and split it into segments, whereafter the audio is attenuated based on which class it is. Requirements were set up to ensure that speech would always be detected correctly, and that the audio would be attenuated when necessary. A system was then developed based on these requirements.

Tests were performed to evaluate the performance of the system. Tests showed that the system was able to detect segments of speech accurately enough to meet the requirements R1, R2, R3, and R4 but not the last requirement R5. Further work should be done to complete this requirement.

In many ways, the project succeeded in what it set out to achieve. Further work on the project could focus on ensuring that the model could be implemented in real-time, improving the speech detection accuracy, and lastly ensuring that it will be compatible with the audio setup found at Nordsøen Oceanarium.

Bibliography

- [1] Nicoleta Diana Radu. *Enhancing Clarity in Underwater Communication Through Sound Separation*. Shared in private communication. 2024.
- [2] Torben Poulsen. *Acoustic Communication. Hearing and Speech*. "DTU Course Lecture Notes". 2005.
- [3] Tom Bäckström. *Speech Coding with Code-Excited Linear Prediction*. 1st. Signals and Communication Technology. Springer, 2017. ISBN: 978-3-319-50202-1.
- [4] et al. Sharma Garima. "Trends in Audio Signal Feature Extraction Methods." In: *Applied Acoustics* (2020). URL: <https://doi.org/10.1016/j.apacoust.2019.107020>.
- [5] Haytham Fayek. *Speech Processing for Machine Learning: Filter banks, Mel-Frequency Cepstral Coefficients (MFCCs) and What's In-Between*. Haytham M. Fayek. URL: <https://haythamfayek.com/2016/04/21/speech-processing-for-machine-learning.html>.
- [6] F. A. Butt M. Jalil and A. Malik. *Short-time energy, magnitude, zero crossing rate and autocorrelation measurement for discriminating voiced and unvoiced segments of speech signals*. URL: <https://ieeexplore.ieee.org/document/6557272>.
- [7] Librosa. *librosa feature spectral rolloff*. Librosa. URL: https://librosa.org/doc/latest/generated/librosa.feature.spectral_rolloff.html.
- [8] et al. Dan-Ning Jiang Lie Lu. *MUSIC TYPE CLASSIFICATION BY SPECTRAL CONTRAST FEATURE*. Microsoft Research Asua. URL: <https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=1035731>.
- [9] Robin D. Morris William A. Sethares and James C. Sethares. *Beat Tracking of Musical Performances Using Low-Level Audio Features*. URL: https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=1395972&getft_integrator=scopus&tag=1.
- [10] Harshita Gupta and Divya Gupta. *LPC and LPCC method of feature extraction in Speech Recognition System*. IEEE. URL: <https://ieeexplore.ieee.org/document/7508171>.
- [11] Sneha Basak and Agrawal et al. "Challenges and Limitations in Speech Recognition Technology: A Critical Review of Speech Signal Processing Algorithms, Tools and Systems". In: *Computer Modeling in Engineering & Sciences* 135 (October 2022), pp. 1–37. DOI: 10.32604/cmes.2022.021755.
- [12] Andrew Senior et al. Sak Haşim. *Long short-term memory recurrent neural network architectures for large scale acoustic modeling*. https://www.isca-archive.org/interspeech_2014/sak14_interspeech.pdf. 2014.

- [13] Mohd Fadzil Hassan et al. Safwan Mahmood Al-Selwi. "RNN-LSTM: From applications to modeling techniques and beyond—Systematic review". In: *Journal of King Saud University - Computer and Information Sciences* (2024). URL: https://www.sciencedirect.com/science/article/pii/S1319157824001575?ref=pdf%5C_download%5C&fr=RR-2%5C&rr=8cd499dc9acdbe60.
- [14] IBM. *What are SVMs?* IBM. URL: <https://www.ibm.com/topics/support-vector-machine>.
- [15] javatpoint. *Support Vector Machine Algorithm*. javatpoint. URL: <https://www.javatpoint.com/machine-learning-support-vector-machine-algorithm>.
- [16] Speech and Language Processing. Daniel Jurafsky & James H. Martin. *Speech and Language Processing*. Stanford university. URL: <https://web.stanford.edu/jurafsky/slp3/A.pdf>.
- [17] IBM. *Feature Selection in Machine Learning*. IBM. URL: <https://www.ibm.com/topics/decision-trees>.
- [18] Ruchi. Doshi. *Machine Learning: Master Supervised and Unsupervised Learning Algorithms with Real Examples*. 1st ed. BPB Publications, 2022. ISBN: 9789391392352.
- [19] JEROME H. FRIEDMAN. "GREEDY FUNCTION APPROXIMATION: A GRADIENT BOOSTING MACHINE." In: *The Annals of Statistics* (2001). URL: <https://www-jstor-org.zorac.aub.aau.dk/stable/2699986>.
- [20] Aurelle Tchagna. *Feature Selection in Machine Learning*. Baeldung. URL: <https://www.baeldung.com/cs/feature-selection>.
- [21] Jason Brownlee PhD. *How to Choose a Feature Selection Method For Machine Learning*. Guiding Tech Media. URL: <https://machinelearningmastery.com/feature-selection-with-real-and-categorical-data/>.
- [22] Gurmeet Kaur. *Filter Methods For Feature Selection in Machine Learning*. almaBetter. URL: <https://www.almabetter.com/bytes/articles/filter-methods-for-feature-selection-in-machine-learning>.
- [23] *Introduction to Feature Selection methods with an example*. Analytics Vidhya. URL: <https://www.analyticsvidhya.com/blog/2016/12/introduction-to-feature-selection-methods-with-an-example-or-how-to-select-the-right-variables/>.
- [24] Soledad Galli. *Feature Selection in Machine Learning*. URL: <https://www.blog.trainindata.com/feature-selection-for-machine-learning/>.
- [25] Rohit Kundu. *F1 Score in Machine Learning: Intro and Calculation*. V7. URL: <https://www.v7labs.com/blog/f1-score-guide>.
- [26] Evidentlyai. *How to interpret a confusion matrix for a machine learning model*. Evidentlyai. URL: <https://www.evidentlyai.com/classification-metrics/confusion-matrix>.

- [27] Daniel Berrar. *Cross-validation*. Data Science Laboratory, Tokyo Institute of Technology. URL: https://www.researchgate.net/profile/Daniel-Berrar/publication/324701535_Cross-Validation/links/5cb4209c92851c8d22ec4349/Cross-Validation.pdf.
- [28] Jason Brownlee PhD. *A Gentle Introduction to k-fold Cross-Validation*. Machine Learning Mastery. URL: <https://machinelearningmastery.com/k-fold-cross-validation/>.
- [29] Trevor Hastie et. al. Gareth James Daniela Witten. *An Introduction to Statistical Learning: with Applications in Python*. 1st. Springer Nature, 2023. ISBN: 9783031387463.
- [30] Jason Brownlee PhD. *LOOCV for Evaluating Machine Learning Algorithms*. Machine Learning Mastery. URL: <https://machinelearningmastery.com/loocv-for-evaluating-machine-learning-algorithms/>.
- [31] SciKit-Learn. 3.1. *Cross-validation: evaluating estimator performance*. SciKitLearn. URL: https://scikit-learn.org/1.5/modules/cross_validation.html.
- [32] SciKit-Learn. 10. *Common pitfalls and recommended practices*. SciKitLearn. URL: https://scikit-learn.org/1.5/common_pitfalls.html.
- [33] Eda Kavlakoglu. Jacob Murel Ph.D. *What is dimensionality reduction?* IBM. 2024. URL: <https://www.ibm.com/topics/dimensionality-reduction>.
- [34] *Introduction to Dimensionality Reduction*. geeksforgeeks. 2023. URL: <https://www.geeksforgeeks.org/dimensionality-reduction/>.
- [35] *Linear Discriminant Analysis*. IBM. 2023. URL: <https://www.ibm.com/topics/linear-discriminant-analysis>.
- [36] *Linear Discriminant Analysis in Machine Learning*. geeksforgeeks. 2024. URL: <https://www.geeksforgeeks.org/ml-linear-discriminant-analysis/>.
- [37] GeeksForGeeks. *Getting Started with Jupyter Notebook*. GeeksForGeeks. URL: <https://www.geeksforgeeks.org/getting-started-with-jupyter-notebook-python/>.
- [38] Brian McFee et al. *Librosa/librosa: 0.10.2.post1*. Zenodo. 2024. DOI: 10.5281/zenodo.11192913. URL: <https://zenodo.org/records/11192913>.
- [39] Scikit-learn. *StandardScaler*. Scikit-learn. URL: <https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.StandardScaler.html>.
- [40] Somayeh Youssefi. *Permutation importance vs impurity-based feature importance*. Medium. URL: <https://medium.com/@syoussefi600/permutation-importance-vs-impurity-based-feature-importance-1c1a8d027479>.
- [41] Soundcraft. *User Guide v.2.8 For Soundcraft Ui12 & Ui16*. Version 2.8. Soundcraft. 2024. 84 pp. URL: https://www.markertek.com/Attachments/Manuals/Soundcraft/UI16-Manual.pdf?srsltid=AfmB0oq084jamKjj6uxh-IbPr-NGUC6PXtUJc8Gpcbar_ErjgBJGh90R.
- [42] Soundweb. *Woundweb London BLU-100 Installation Guide*. Soundweb. 2024. 16 pp. URL: <https://www.pdfmanualer.dk/bss-audio/soundweb-london-blu-100/manual>.

- [43] The Editors of Encyclopædia Britannica. *phoneme*. Encyclopædia Britannica. URL: <https://www.britannica.com/topic/phoneme>.
- [44] Hao Huang. *Leveraging Phone Mask Training for Phonetic-Reduction-Robust E2E Uyghur Speech Recognition*. Xinjiang University. URL: https://www.researchgate.net/publication/352355866_Leveraging_Phone_Mask_Training_for_Phonetic-Reduction-Robust_E2E_Uyghur_Speech_Recognition.